The Australian National University

School of Computing

JHQ-GPU

Representation-Aware GPU ANN Search and Workload-Level Refinement Calibration

Rui Luo

Supervisor: Dr Mengxuan Zhang

A research thesis submitted for

COMP8800 — Advanced Computing Research Project

October 2026

Abstract

Approximate nearest-neighbour (ANN) search supports similarity retrieval over large collections of embedding vectors. GPUs accelerate this search, but limited device memory makes compressed indexes attractive. JHQ is a hierarchical quantisation method that screens candidates with compact codes and refines a smaller set using residual codes. This thesis develops JHQ-GPU to study how this representation can support efficient GPU construction and search.

The implementation combines GPU model fitting and batched encoding with bounded input staging and a packed, list-ordered layout. Its scan replaces each 256-entry distance table with two 16-entry tables, preserving scores in real arithmetic. Grouped scanning preserves exact global candidate selection under a fixed ordering of computed scores and positions. Calibration chooses how many candidates to refine without ground-truth labels, by comparing returned neighbours with a larger-budget reference.

Evaluation on an NVIDIA RTX 5090 compares methods on Vogue, arXiv, OpenAI-1536 and OpenAI-3072. At recall 0.97 and batch 1,000, JHQ-GPU delivers 1.32–1.75× the host-to-host throughput of the CAGRA fp32 graph index across all four datasets and is the fastest tested method on three. For the larger BGE and Stella text-embedding datasets, whose raw FP32 vectors exceed device memory, JHQ-GPU enables single-GPU search; the largest index contains 17.78 million vectors and uses 20.44 GiB of resident GPU memory.

Acknowledgements

I thank my supervisor, Dr Mengxuan Zhang, for proposing the research direction and providing guidance through regular discussions throughout the project.

Research and AI Assistance

I used OpenAI Codex for substantive drafting, revision, LaTeX, analysis and code/log checks [30]; ChatGPT for planning and reviews [29]; and Anthropic Claude for implementation, experiments and review [4]. Selected prompts and usage details are available in the AI-use record. I am responsible for the claims in this thesis.

Contents

Abstract i

Acknowledgements i

Research and AI Assistance i

Notation and Abbreviations vi

1 Introduction 1
- 1.1 Background and Motivation . . . 1
- 1.2 Problem Definition and Scope . . . 1
- 1.3 Technical Challenges . . . 1
- 1.4 Research Questions . . . 2
- 1.5 Research Contributions . . . 2

2 Background and Related Work 4
- 2.1 Distance and Recall . . . 4
- 2.2 IVF, PQ, ADC and Refinement . . . 4
- 2.3 JHQ Representation and Assumptions . . . 4
- 2.4 GPU Execution Concepts . . . 6
- 2.5 Graph-Based ANN Indexes . . . 6
- 2.6 Compressed Search and Adaptive Refinement . . . 6
- 2.7 Relation to Prior Work . . . 7

3 Research Design and Measurement 8
- 3.1 Experimental Comparisons and Controls . . . 8
- 3.2 Datasets, Queries, Ground Truth and Splits . . . 8
- 3.3 Timing and Memory Measurement . . . 9
- 3.4 Baselines and Frontier Construction . . . 10
- 3.5 Experimental Records and Reproducibility . . . 11

4 GPU Index Design and Construction 13
- 4.1 Build and Search Workflow . . . 13
- 4.2 Training, Rotation and Codebook Generation . . . 13
- 4.3 IVF Assignment, IDs and Physical Memory Layout . . . 15
- 4.4 Resident Storage, Workspace and Index Lifecycle . . . 17
- 4.5 Construction Cost and Tested Scale . . . 18
- 4.6 Index Representation and Validation . . . 19

5 Representation-Aware GPU Search 21
- 5.1 Query Workflow and Ranking Scores . . . 21
- 5.2 Exact Cartesian LUT Factorisation . . . 22
- 5.3 Packed Codes and Candidate-Parallel Memory Access . . . 24
- 5.4 Exact Global Primary Top-c via Multi-Block Scanning . . . 25
- 5.5 Shared Complete-Key Thresholds and Execution Scheduling . . . 28
- 5.6 Lane-Cooperative Residual Scoring and Final Top-k . . . 30
- 5.7 IVF Probe Selection . . . 31

6 Workload-Level Budget Calibration 33
- 6.1 Refinement Budget and Search Cost . . . 33
- 6.2 Output Agreement and Budget Selection . . . 33
- 6.3 Budget Search and Monotonicity Conditions . . . 35
- 6.4 Budget Selection Baselines . . . 36

6.5 Calibration Quality and Reuse Cost . . . 37
6.6 Calibration Reuse and Applicability . . . 37

7 Experimental Evaluation **38**
7.1 Experimental Protocol . . . 38
7.2 Cross-System Recall–Throughput Comparison . . . 38
7.3 Cartesian LUT, Packed Loads and Lane Refinement . . . 41
7.4 Selection Correctness and Performance . . . 43
7.5 Held-out Budget Quality, Throughput and Payback . . . 44
7.6 Batch Size, Scheduling and Workload Imbalance . . . 48
7.7 Cold Construction, Resident Memory and Tested Scale . . . 49
7.8 Numerical Checks and Recall Analysis . . . 51

8 Discussion and Future Work **53**
8.1 Design Implications . . . 53
8.2 Alternative Scoring and Coding Experiments . . . 53
8.3 Threats to Internal and External Validity . . . 55
8.4 Future Research Directions . . . 55
8.5 Deployment Considerations . . . 55

9 Conclusion **56**
9.1 Answers to the Four Research Questions . . . 56
9.2 Summary and Outlook . . . 56

Bibliography **57**

List of Figures

2.1 Candidate and scoring cascade . . . 5
3.1 Host and device timing boundaries . . . 10
4.1 Build and query architecture . . . 13
4.2 Physical GPU index layout . . . 16
4.3 One- and two-pass construction . . . 18
4.4 Separate construction-allocation lifetimes . . . 19
5.1 Query stages and selection boundaries . . . 21
5.2 Cartesian primary-LUT factorisation . . . 23
5.3 Packed candidate-parallel access . . . 24
5.4 Local coverage and truncated pairwise merging . . . 27
5.5 Shared complete-key filtering . . . 29
5.6 Sixteen-lane residual refinement . . . 31
6.1 Work affected by the refinement budget . . . 33
6.2 Workload-level calibration and evaluation . . . 34
6.3 Nested and non-nested candidate pools . . . 36
7.1 E1 recall–throughput frontiers . . . 39
7.2 TF32 and FP32 precision frontiers . . . 40
7.3 Small-batch competitive regions . . . 41
7.4 Cartesian lookup-table grouping . . . 42
7.5 Packed access and refinement controls . . . 43
7.6 Held-out risk across sample sizes . . . 46
7.7 Finite-pool risk prediction and observation . . . 47
7.8 Direct cumulative calibrated-reuse time . . . 47
7.9 Batch-dependent execution regimes . . . 48
7.10 Recorded construction costs . . . 49
7.11 Resident-memory decomposition . . . 50
7.12 Large-dataset search frontiers . . . 51
8.1 Mailman and rank-oriented coding . . . 54

List of Tables

1.1 Technical contributions and supporting evidence . . . 3
2.1 Related approaches and JHQ-GPU design questions . . . 7
3.1 Research questions and experimental controls . . . 8
3.2 Dataset and training configuration . . . 9
3.3 Principal E1 tuning dimensions . . . 10
3.4 Component implementations . . . 11
4.1 Per-vector storage model . . . 17
4.2 Analytical row-array occupancy by construction phase . . . 18
4.3 Original-input FP64 recoding audit . . . 20
4.4 Representation contracts and audit coverage . . . 20
5.1 Query-stage selection contracts . . . 22
5.2 Cartesian lookup-table trade-offs . . . 23
6.1 Budget controls and their roles . . . 36
7.1 Cross-system comparison at recall 0.97 . . . 38
7.2 FP32 sensitivity at matched configurations . . . 39
7.3 Same-session precision anchors . . . 40
7.4 Small-batch matched-recall ratios . . . 41
7.5 Refinement gains at 128 probes . . . 42
7.6 Strict computed-score selection checks . . . 43
7.7 Probe-selection throughput ratios . . . 44
7.8 Full-pool budget calibration . . . 45
7.9 Sampled-budget held-out quality . . . 45
7.10 Sampled-calibration economics . . . 46
7.11 Direct costs for 500 reuse batches . . . 47
7.12 Integrated execution and scheduling gains . . . 48
7.13 Cold construction and resident memory . . . 49
7.14 E1 allocation ranges . . . 50
8.1 Threats to validity . . . 55

Notation and Abbreviations

Symbol	Meaning
N, d	Number of database vectors and original dimension
M, D_s	Number of primary subspaces and dimensions per subspace
$a_{\text{add}}, a_{\text{in}}, a_{\text{assign}}$	Configured add, staged input, and independently allocated assignment rows
n_b, S_{stage}	Pipeline buffer count and byte budget per input/rotated array
Π	Square orthogonal transform; $q' = \Pi q$
B, B_r	Primary bits per subspace and residual bits per coordinate
$\rho(p), o_\ell$	Input ID at physical position p, and start of IVF list ℓ
k	Number of returned neighbours
n_c, c	Routed candidates and primary survivors receiving refinement
α	Refinement multiplier, with $c = \max\{k, \lceil \alpha k \rceil\}$
S, ϵ	Sample count; allowed missing-reference slots
$C_\alpha, \mathcal{R}_\alpha$	Valid survivors; returned IDs at budget α
$\alpha_{\text{ref}}, B_{\text{reuse}}$	Reference refinement budget and number of subsequent reuse batches
g_{LUT}	Number of groups in a factorised lookup table
g_{scan}	Number of scan groups per query; zero labels the one-block path
$C_{\text{buf}}, T_{\text{blk}}$	Buffer capacity; threads per block
$n_{\text{LUT}}(g_{\text{LUT}})$	LUT entries per primary subspace
$\kappa(D, p)$	Packed full key for bounds
ℓ_{ref}	Lanes cooperating on one survivor
D_{raw}	$\|q - x\|_2^2$
D_{P}	$\|q' - \widehat{p}\|_2^2$
D_{H}	$\|q' - \widehat{p} - \widehat{r}\|_2^2 + \|q'\|_2^2$

Abbreviation	Meaning
ANN	Approximate nearest neighbour
ADC	Asymmetric distance computation
IVF	Inverted file
JHQ	Johnson–Lindenstrauss enhanced hierarchical quantisation
LUT	Lookup table
PQ	Product quantisation
QPS	Queries per second
GT	Ground truth
FP32	IEEE binary32; GEMM mode stated separately
TF32	Reduced-precision Tensor Core inputs; FP32 accumulation
CV	$\text{sd}(n_c)/\,\text{mean}(n_c)$

1 Introduction

1.1 Background and Motivation

Vector search returns nearby embeddings from a large database. GPU parallelism can accelerate distance calculations, but raw high-dimensional vectors can exceed device memory. Compression can make larger indexes resident, but search still has to read codes and select promising vectors. JHQ is a hierarchical quantisation method that first compares compact primary codes to form a shortlist, then uses residual codes for a more detailed comparison [16]. This thesis studies how to execute these stages efficiently on a GPU while respecting their scoring and selection rules, and measures the resulting quality–speed trade-off.

Graph-based GPU indexes store vectors and graph links; inverted-file (IVF) indexes store the representations searched within selected database partitions [18, 28, 33]. If these structures exceed device memory, the complete index cannot remain resident. Our largest benchmark, Stella, contains 17.78 million 1,024-dimensional text embeddings. Its raw FP32 storage, available device memory and JHQ-GPU resident allocation, respectively, illustrate the capacity constraint (section 7.7):

$$S_{\text{raw}} = 67.81\,\text{GiB} > S_{\text{GPU}} \simeq 31.8\,\text{GiB} > S_{\text{resident}} = 20.44\,\text{GiB}.$$

This motivates a GPU-resident index that searches compressed codes instead of keeping the full FP32 database on the device.

1.2 Problem Definition and Scope

Given a vector database $\mathcal{X} \subset \mathbb{R}^d$ and query q, the task is to return k approximate nearest neighbours under squared Euclidean distance. The research problem is to construct and search JHQ's compressed index within a single GPU's memory, and improve query throughput at a given recall target.

Search first selects relevant IVF lists, screens their vectors using primary codes, then uses residual codes to rescore a smaller shortlist. The refinement budget controls how many candidates enter this last stage. Ground truth (GT) gives exact neighbours in the original vector space; recall is the fraction of these neighbours recovered.

The study retains JHQ's dimension-preserving orthogonal transform and hierarchical codes (section 2.3), and investigates GPU construction, data layout, search execution and refinement-budget selection. Calibration compares smaller-budget outputs with a larger-budget reference without GT, then reuses one chosen multiplier for a fixed index and an assumed stable query distribution. GT evaluation checks the resulting recall.

Evaluation uses one RTX 5090 and squared-L2 top-10. The main cross-system comparison uses four datasets and batch 1,000; lookup-table, memory-layout and budget tests cover six datasets. A small-batch study adds batches 8/32 (section 7.2.6). The index and core search computation remain on the GPU; preparing and transferring queries within the search call are included in its timing. Dynamic updates, host paging and multi-GPU execution are outside the implementation; billion-scale search is not evaluated.

1.3 Technical Challenges

Moving JHQ to a GPU raises three technical challenges.

First, compressed codes reduce storage but can still be expensive to scan. Each eight-bit primary code addresses a 256-entry distance lookup table (LUT); these tables must be built and may be too large for fast on-chip memory. Because JHQ combines per-coordinate quantisation levels, this table can be split exactly in real arithmetic into two 16-entry tables, one for each

four-bit half of the byte. The challenge is whether the smaller tables repay the extra lookups and additions on the GPU.

Second, small batches can contain too few queries to keep the GPU busy. Splitting each query's candidates across thread blocks creates parallel work, but equal local quotas can discard global winners if the best candidates fall in one block. One solution is to let each group retain its own full shortlist before merging, at the cost of memory and merging work. The challenge is to retain all potential winners while keeping this overhead below the parallelism gain. This thesis studies shared cutoffs and batch-dependent scheduling as ways to reduce that overhead.

Third, the refinement work needed for a target recall varies by workload. Too few candidates can discard true neighbours; too many spend time reading and scoring residual codes without improving the answers. The aim is to choose a reusable budget without ground-truth labels. A larger-budget search provides a reference, but can itself miss true neighbours. Agreement with it therefore needs separate checks of recall on test queries, alongside the cost of calibration and subsequent search.

1.4 Research Questions

This thesis asks how representation-aware execution and workload-level refinement calibration can make JHQ an efficient, memory-resident GPU ANN system. Representation-aware execution exploits the code structure in data layout and scoring; workload-level calibration chooses one reusable refinement budget from unlabelled queries. Construction and capacity establish the system's foundation, while cross-system comparisons assess its practical value. Four research questions follow.

RQ1: GPU index construction and capacity. Can a complete JHQ index be built and kept resident on one GPU, and at what construction time, peak memory and resident capacity (section 7.7)?

RQ2: Representation-aware execution. How can JHQ's code structure guide GPU table lookup, memory access and candidate selection to improve throughput while maintaining each stage's specified ranking and tie rules (sections 7.3, 7.4 and 7.6)?

RQ3: Workload-level refinement calibration. Can a refinement budget calibrated once per workload without ground-truth labels keep recall on separate test queries close to that of a larger-budget search and repay its calibration cost (section 7.5)?

RQ4: Cross-system search performance. Under host-to-host timing, at which recall targets and batch sizes does JHQ-GPU outperform the tested GPU ANN baselines (section 7.2)?

Chapter 3 turns these questions into controlled comparisons. Together they separate what was built, whether its execution is correct, what effort a workload requires, and whether the resulting system has useful operating regions.

1.5 Research Contributions

The contributions are a GPU index-construction pipeline, search execution designed around JHQ's codes, and workload-level refinement calibration. Table 1.1 distinguishes these additions from JHQ's quantisation method and established GPU algorithms, which are prior work.

Table 1.1. Technical contributions and supporting evidence.

	Inherited foundation	Project contribution	Evidence
C1	JHQ's transform and hierarchical codes	GPU model fitting/encoding, batched construction and codes arranged in list order	Search of 17.78M vectors at 20.44 GiB resident; fitting and encoding 10.54–10.65 s; sampled construction peak 22.58 GiB
C2	JHQ code structure and existing GPU algorithms	Two-table lookup, efficient code reads, parallel selection with shared cutoffs and cooperative scoring	Two-table lookup leads in 22/24 settings; scheduling gives 2.727× single-block throughput at batch 8; grouped-selection tests match CPU sorting of the same scores
C3	Candidate expansion and output comparison	An unlabelled budget rule, its search conditions, and quality/cost evaluation on separate queries	Using the whole calibration set: at most 0.0006 recall loss versus the best tested budget in 12/12 settings; direct reuse timing in four settings

Individual contribution. My supervisor, Dr Mengxuan Zhang, proposed the direction of bringing JHQ to GPUs and provided guidance through regular discussions. I carried out the work summarised in table 1.1, including its experimental design, baseline comparisons and analysis, and decided which approaches and conclusions to retain. AI tools provided substantial assistance, as detailed in the research and AI assistance statement.

2 Background and Related Work

This chapter introduces the search objective, compressed representations and GPU execution concepts used throughout the thesis. It then positions JHQ-GPU against graph indexes, quantised search and adaptive refinement.

2.1 Distance and Recall

For the search task introduced in Chapter 1, squared Euclidean distance is

$$D_{\text{raw}}(q, x) = \|q - x\|_2^2. \tag{2.1}$$

Evaluation uses the ten stored ground-truth (GT) neighbour IDs for each query. Ties follow this fixed GT prefix (equation (3.1)).

2.2 IVF, PQ, ADC and Refinement

An inverted-file (IVF) index partitions the database into lists, usually by assigning each vector to a nearby centroid. A query searches n_{probe} selected lists. If $\mathcal{L}(q)$ is their index set, the vectors available for scoring are

$$U(q) = \bigcup_{\ell \in \mathcal{L}(q)} \text{list}_\ell, \qquad n_c = |U(q)|.$$

Any true neighbour outside $U(q)$ is excluded before vector scoring begins.

Product quantisation (PQ) compresses each vector by dividing its coordinates into M subspaces and learning a codebook for each [17]. In subspace m, the code z_m identifies the representative vector $\mathcal{C}_m[z_m]$; together these representatives reconstruct $\widehat{x}$. Asymmetric distance computation (ADC) keeps the query unquantised and evaluates

$$\|q - \widehat{x}\|^2 = \sum_{m=0}^{M-1} \|q_m - \mathcal{C}_m[z_m]\|^2. \tag{2.2}$$

Here q_m is the query's subvector. Distances to codewords are tabulated once per query, so scanning database codes uses lookups. PQ separates subspaces, but an arbitrary learned codebook need not separate coordinates *within* a subspace. JHQ's primary codebook has this additional structure.

Refinement spends more work on a shortlist selected by an initial score [10, 18]. Let α be the refinement multiplier and $c = \max\{k, \lceil \alpha k \rceil\}$ the requested shortlist size. For $|U(q)| \geq k$, write $C_\alpha(q)$ for up to c retained candidates and $\mathcal{R}_\alpha(q)$ for the final k returned vectors. Then

$$\mathcal{R}_\alpha(q) \subseteq C_\alpha(q) \subseteq U(q) \subseteq \mathcal{X}. \tag{2.3}$$

If $\mathcal{T}_k$ is the GT set for the query, inclusion gives

$$|\mathcal{R}_\alpha \cap \mathcal{T}_k| \leq |C_\alpha \cap \mathcal{T}_k| \leq |U \cap \mathcal{T}_k|.$$

Increasing probes expands the routed pool; increasing α expands the requested shortlist. To attribute recall loss to routing, shortlisting or reconstruction, each stage must be compared while holding the others fixed. The next section defines JHQ's two scoring stages.

2.3 JHQ Representation and Assumptions

2.3.1 Rotation and hierarchical codes

JHQ first applies a square orthogonal transform $\Pi \in \mathbb{R}^{d \times d}$ to both database vectors and queries [16]. Write $y = \Pi x$ and $q' = \Pi q$. In real arithmetic,

$$\begin{aligned} \Pi^\top \Pi = I \implies \|\Pi(q - x)\|^2 \\ = (q - x)^\top \Pi^\top \Pi (q - x) = \|q - x\|^2. \end{aligned} \tag{2.4}$$

Thus rotation preserves dimension and distances before quantisation. Finite-precision matrix multiplication is evaluated separately in Chapter 4.

The primary code gives a coarse reconstruction $\widehat{p}(x)$ of y. In the evaluated configuration, each eight-coordinate subspace uses one byte: each coordinate takes one of two levels, $-a$ or $+a$, so the byte records eight signs. More generally, M subspaces have dimension $D_s = d/M$, with B bits selecting one of $K = 2^B$ combinations of $L = 2^{B/D_s}$ scalar levels. For supported parameters with integer L, JHQ uses the Cartesian product

$$a_i = \sigma\Phi^{-1}\left(\frac{i-1/2}{L}\right), \quad i = 1,\dots,L, \qquad \mathcal{C}_m = \{a_1,\dots,a_L\}^{D_s}. \tag{2.5}$$

Here Φ is the standard normal cumulative distribution function and σ is a scale estimated from vector energy. At $B = D_s = 8$, this gives $L = 2$, $\mathcal{C}_m = \{-a, +a\}^8$ and $a \simeq 0.67449\sigma$. Unit-normalised inputs motivate a scale near $1/\sqrt{d}$. The transform and hierarchical representation are inherited from JHQ.

Residual coding approximates the remaining error, $r = y - \widehat{p}$, by $\widehat{r}$, giving $\widehat{y} = \widehat{p} + \widehat{r}$. The evaluated residual quantiser learns scalar levels shared across coordinates within each subspace and uses one byte per coordinate. Each vector therefore stores $d/8$ primary bytes and d residual bytes before metadata. A smaller refinement budget reduces residual reads, while these stored codes remain unchanged.

This residual is relative to the primary reconstruction. By contrast, residual-coded IVF-PQ subtracts the centroid $\mu_{\ell(x)}$ of the list containing x:

$$r_{\text{IVF-PQ}} = x - \mu_{\ell(x)}, \qquad r_{\text{JHQ}} = \Pi x - \widehat{p}(x).$$

JHQ uses IVF for routing; its reconstructed vector does not add an IVF centroid.

2.3.2 Primary and refined scoring

Primary screening compares q' with the coarse reconstruction:

$$D_{\text{P}} = \|q' - \widehat{p}\|^2.$$

Only shortlisted candidates are rescored with both primary and residual codes. The implemented refined score is

$$D_{\text{H}} = \|q' - \widehat{p} - \widehat{r}\|^2 + \|q'\|^2. \tag{2.6}$$

The extra term $\|q'\|^2$ is constant across candidates for one query, so the score orders reconstructed vectors by squared distance. It need not reproduce the raw-vector ordering used for GT. The score expansion appears in equation (5.20).

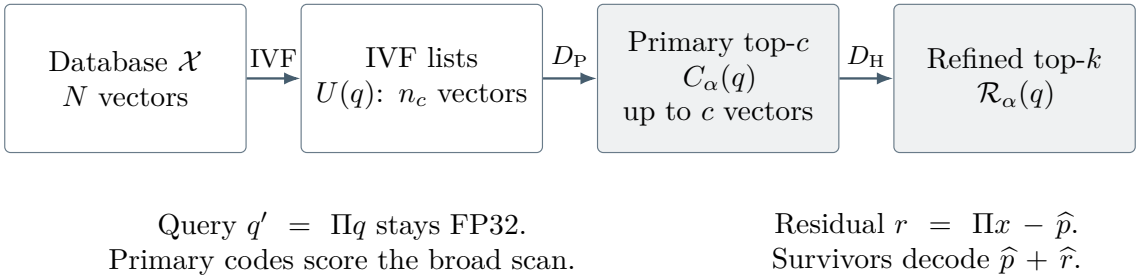


Figure 2.1. Candidate and scoring cascade in JHQ. IVF selects lists, primary scores form a shortlist, and residual codes refine that shortlist before the final top-k selection.

With n_c routed vectors, M primary subspaces and up to c survivors, the score work is

$$W_{\text{primary}} = O(n_c M) \text{ lookups}, \qquad W_{\text{refine}} = O(cd) \text{ coordinates}.$$

Memory access and candidate selection also contribute to search time.

2.3.3 Modelling assumptions

Random rotation can spread coordinate energy, motivating the Gaussian-inspired levels above. Distance preservation does not imply independent, zero-mean Gaussian coordinates: a fixed transform maps a database mean μ to $\Pi\mu$, with $\|\Pi\mu\| = \|\mu\|$. Chapter 8 examines centring as an alternative coding choice.

The primary level is a Gaussian mid-quantile, rather than the level that minimises reconstruction mean-squared error (MSE). For $X \sim \mathcal{N}(0, \sigma^2)$ and a zero decision threshold, the latter is

$$\begin{aligned} a_{\mathrm{MSE}} &= \arg\min_a \mathbb{E}[(X-a)^2 \mid X > 0] \\ &= \mathbb{E}[X \mid X > 0] = \sigma\sqrt{2/\pi} \simeq 0.79788\sigma. \end{aligned}$$

These values serve different objectives. For fixed sign codes, any positive common primary scale preserves primary ranking (equation (4.3)); residual reconstruction still depends on the scale. Thus the algebraic scoring properties do not establish optimal quantisation or raw-vector recall.

2.4 GPU Execution Concepts

GPU work is organised into thread blocks; threads within a block can cooperate through shared memory. A warp is a group of 32 threads scheduled together. Registers, shared memory and limits on resident blocks constrain how much work can run concurrently [25].

Search can distribute work across queries, candidates and vector coordinates.

Candidate selection keeps the smallest scores from the scanned pool. Faiss uses warp-level selection [18]; other GPU top-k methods distribute selection over larger arrays [39]. Chapter 5 explains the selection and scheduling used by JHQ-GPU.

2.5 Graph-Based ANN Indexes

Graph indexes connect nearby vectors and navigate these links to find promising candidates, rather than scanning all vectors in selected lists. HNSW uses a hierarchy of proximity graphs for coarse-to-fine navigation [21]. On GPUs, the challenge is to expose parallel work during traversal. SONG studies parallel graph search [40]; CAGRA designs graph construction and search for GPUs [28]. CAGRA is the resident graph baseline here, tested with both FP32 and quantised vectors.

Other designs extend capacity across storage boundaries. DiskANN combines graph search with SSD storage for databases larger than main memory [34]; BANG combines a host-resident graph with compressed GPU scoring [38]. Their capacity gains depend on communication across storage boundaries.

2.6 Compressed Search and Adaptive Refinement

Optimised product quantisation (OPQ) learns a rotation and subcodebooks to reduce reconstruction error [13]. ScaNN's anisotropic quantisation instead targets inner-product approximation [15]. The distinction matters because minimising vector reconstruction error need not best preserve neighbour ordering.

RaBitQ estimates distances using randomised binary codes [11], with a multi-bit extension offering a finer representation [12]. GPU IVF-RaBitQ combines compressed distance estimation with IVF routing and fused search [33]. It provides a direct compressed-search baseline for testing whether JHQ's additional residual scoring is worthwhile.

Fast Scan and Quicker ADC accelerate CPU distance computation using small tables accessed through single-instruction, multiple-data (SIMD) operations [1, 2]. Table size and hardware lookup support are therefore central to speed. JHQ-GPU splits tables using its

primary codebook's coordinate structure; Chapter 5 analyses the identity and finite-precision effects.

Faiss and the GPU search library cuVS expose parameters controlling the number of candidates refined [7, 10]. Faiss autotuning uses labelled evaluation [9]; learned early termination predicts when to stop [19]; DARTH expresses search effort through recall targets [6]. These approaches provide the comparison points for the label-free calibration in Chapter 6.

2.7 Relation to Prior Work

Table 2.1 relates the preceding methods to JHQ-GPU. Small-table methods motivate exploiting code structure; graph and binary-code indexes provide alternative speed–capacity trade-offs; adaptive-effort methods provide the quality-control comparison. Chapters 4–6 develop these design choices, and Chapter 7 evaluates them.

Table 2.1. Related approaches and the design questions they motivate for JHQ-GPU.

Approach	Main trade-off	Question for JHQ-GPU
IVF-PQ / OPQ	Learned codebooks reduce distortion but need not separate coordinates internally	How can the primary code's coordinate structure simplify GPU scoring?
Fast Scan / Quicker ADC	Small tables accelerate lookup under hardware and precision constraints	Does splitting tables repay extra GPU lookups?
SONG / CAGRA	Graph navigation reduces scoring but adds graph storage and dependent access	Where can a compressed scan compete with resident graph search?
BANG	Host storage extends capacity at the cost of communication	What can a complete compressed index resident on one GPU achieve?
IVF-RaBitQ	Code size and probe effort balance distance-estimation quality and speed	When does residual refinement justify its extra work?
Adaptive effort	Quality control may require labels, prediction or calibration	Can workload agreement select a useful budget at acceptable quality and cost?

3 Research Design and Measurement

Paired execution, held-out calibration and system frontiers use different controls, defined here.

3.1 Experimental Comparisons and Controls

Table 3.1 defines the comparison unit and what would challenge each research claim. A *cell* fixes a dataset, index configuration and workload setting; arms are alternative implementations or budgets within that cell. For paired execution measurements, the index state, queries, probe count, refinement budget and scoring representation are held fixed unless the named treatment changes them. The recorded paired ratio is $Q_{\text{treatment}}/Q_{\text{control}}$. Across J stated cells, equal weighting gives

$$G = \exp\left(\frac{1}{J}\sum_{j=1}^{J}\log\frac{Q_{\text{treatment},j}}{Q_{\text{control},j}}\right).$$

Table 3.1. Research questions as testable comparisons. A capacity observation has a different control from a paired execution intervention.

Question	Intervention / comparison	Held fixed or accounted for	Measure and counter-evidence
RQ1: build and capacity	GPU construction; cached versus cold lifecycle; compressed versus raw storage	N, d, representation, routing and timed construction boundary	Train/add time and resident bytes; failed builds or model–allocation gaps delimit feasibility
RQ2: execution	LUT grouping, word access, refinement lanes, grouped scan, shared bound and probe selection	Paired state, workload and score; reference output semantics	Wall QPS, stage time and output checks; slowdowns or non-tie mismatches challenge the mechanism
RQ3: budget	Sampled / full-pool rule versus fixed 64, 100, 200, enumeration and GT oracle	One index and route; disjoint calibration and held-out query positions	Held-out recall loss, selected-arm QPS and calibration cost; threshold crossings challenge reliability
RQ4: operating region	Independently tuned JHQ, CAGRA, IVF-PQ and IVF-RaBitQ	Exported vectors, GT, k, device and host-to-host boundary; memory reported separately	Matched-recall QPS and coverage; stronger competitors at other recall targets challenge broad superiority

Ablations estimate speed effects, not unmeasured cache/occupancy changes. System comparisons are separate (section 7.6).

3.2 Datasets, Queries, Ground Truth and Splits

Table 3.2 records the six datasets used in this study. The main cross-system comparison (E1) uses the first four; budget, LUT and scale tests cover all six. Vogue contains fashion-image embeddings; arXiv contains abstract embeddings; the two OpenAI datasets use Qdrant's DBpedia exports [5, 31, 32, 35]. BGE uses the Italian Wikipedia configuration, and Stella uses the TREC24 BioGen corpus export [36, 37]. Sizes describe the measured exports after preparation, rather than the current upstream cards.

Table 3.2. Data and JHQ training configuration. n_q is the actual query count; $n_t = 39n_{\text{list}}$ is the requested coarse-training sample size. E1 uses the first four rows. All rows use squared L2 and $k = 10$.

Dataset	N	d	n_q	n_{list}	n_t
Vogue	932,328	768	1,000	4,096	159,744
arXiv	2,253,000	768	1,000	8,192	319,488
OpenAI-1536	999,000	1,536	1,000	8,192	319,488
OpenAI-3072	999,000	3,072	1,000	8,192	319,488
BGE-M3	10,091,524	1,024	1,000	32,768	1,277,952
Stella	17,776,615	1,024	999	32,768	1,277,952

OpenAI/BGE/Stella preparation removes non-finite rows and uses

$$x \leftarrow x/\|x\| \quad (\|x\| > 10^{-12}).$$

Seed 42 samples without replacement. Stella uses 999 corpus queries, replacing the 65-query benchmark; relevance labels are absent. OpenAI checks support alignment for all queries/10,299 base rows without original-ID joins (sections 7.2 and 7.2.3).

Methods share exported query/GT files. Exhaustive tiled squared-L2 search over FP32 vectors maintains GT top-k, including its ties:

$$\text{Recall}\,@k = \frac{1}{|\mathcal{Q}|k} \sum_{q \in \mathcal{Q}} |\mathcal{R}_k(q) \cap \mathcal{T}_k(q)|. \tag{3.1}$$

Here $\mathcal{T}_k$ is the stored GT prefix and $\mathcal{R}_k$ the returned ID set. GT widths are 100 (Vogue/arXiv) or 20 (others); only ten entries count. Duplicate IDs earn no extra credit. Range/duplicate/repeat checks are separate.

JHQ uses $M = d/8$, $B = B_r = 8$, eight coarse iterations and 100,000 residual-training rows. Sweeps reload/re-encode; cold builds refit. Training uses database rows.

$$\mathcal{Q}_{\text{cal}} = \{q_i : i \bmod 2 = 0\}, \qquad \mathcal{Q}_{\text{hold}} = \{q_i : i \bmod 2 = 1\},$$

$$(|\mathcal{Q}_{\text{cal}}|, |\mathcal{Q}_{\text{hold}}|) = \begin{cases} (500, 499), & \text{Stella}, \\ (500, 500), & \text{otherwise}. \end{cases}$$

Seeded subsamples of sizes 32, 64 and 128 are drawn 64 times per cell; FULL uses all 500 calibration queries. Labels assess the oracle and holdout; agreement uses outputs (sections 3.5.2 and 6.2).

3.3 Timing and Memory Measurement

The measured interval is the host search call:

$$t_{\text{call}} = t_{\text{return}} - t_{\text{entry}}, \quad \begin{cases} \text{entry: FP32 queries in host memory,} \\ \text{return: host IDs/scores after stream completion.} \end{cases}$$

Staging, transfers, query work and int8 query quantisation are timed; build/fitting are excluded (figure 3.1). Handles/outputs are reused. JHQ uses pageable/C++, CAGRA/PQ pinned/Python, RaBitQ pinned/C++; these costs matter to small margins.

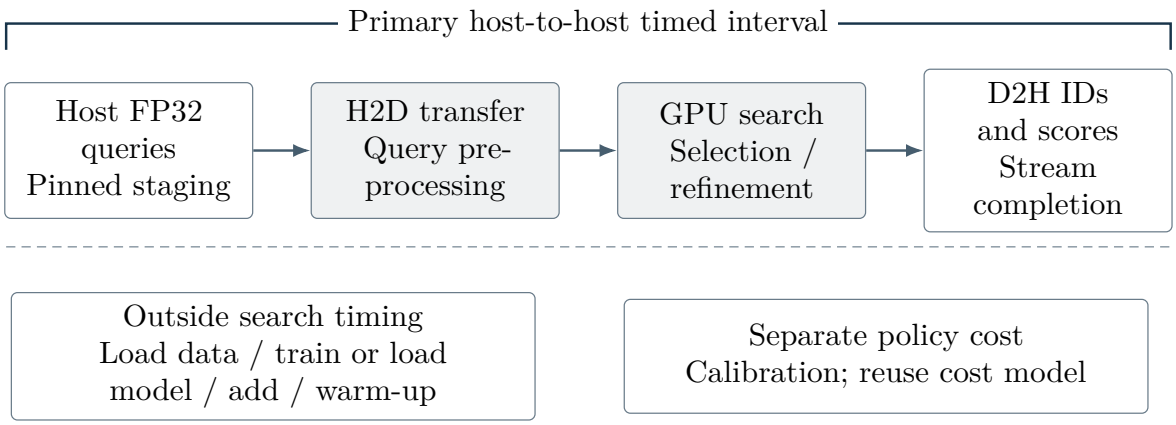


Figure 3.1. Timing boundaries. The host interval includes transfers and query-specific computation. Device-stage profiles explain parts of that interval; separate profiling and synchronisation can change execution, so their sum is not an interchangeable end-to-end latency.

Times are in seconds:

$$\bar{t}_r = \frac{1}{m_r}\sum_{i=1}^{m_r} t_{ri}, \qquad Q_r = \frac{b}{\bar{t}_r}, \qquad Q = \mathrm{median}_r\, Q_r. \tag{3.2}$$

E1: batch 1,000, five rounds, normally 24 calls. JHQ warms once per variant/round; baselines three times initially. Slow baseline points use fewer calls. Just-in-time (JIT) compilation and CUDA Graph capture are excluded; call percentiles do not describe concurrent requests.

Slow valid calls stay. For illustrative call times $(1, 1, 10)$ ms,

$$\bar{t} = 4\,\mathrm{ms} \neq \mathrm{median}(t) = 1\,\mathrm{ms}, \qquad b/\bar{t} \neq b/\,\mathrm{median}(t).$$

Shared indexes/pools do not provide independent builds.

RTX 5090/32,607 MiB; driver 595.71.05; CUDA 13.0; cuVS 26.08.01. OpenMP uses 16 of 25 CPUs; E1 records 575 W. Clocks/load can vary. JHQ supplies compute_80 PTX intermediate GPU code for driver compilation [27]; library builds/device profiles are separate.

Resident totals include model, workspace and context. CAGRA index deltas and JHQ context-subtracted totals differ (sections 4.4 and 7.7). Pools/scratch prevent peak matching. Build peaks use 20 ms sampling; train/add excludes loading/host setup.

Supplementary sessions. Campaigns are separate (section 3.5.2); cached-state/precision changes follow sections 4.2 and 7.2.3. Historical baseline ratios remain cross-session.

3.4 Baselines and Frontier Construction

CAGRA fp32/int8, IVF-PQ and IVF-RaBitQ [17, 28, 33] use separate grids (table 3.3); FP32 brute force is exhaustive. Storage/reconstruction fidelity may differ. Consequently the main frontier measures achievable speed at a quality target within these grids. A separate memory account accompanies it.

Table 3.3. Principal E1 tuning dimensions. Exact per-point parameters and coverage extensions remain in the raw records. The grid is finite and differs between methods.

Family	Recorded search dimensions
JHQ	Probes and α; initial probes 64/128/256 and 0.5/1/2 times the workload budget; coverage extensions reach probe 1,024 and larger/smaller budgets
CAGRA fp32/int8	Graph degree 32/64; search width 1/2/4/8; intermediate top-k capacity 32–512
IVF-PQ	Eight-bit subcodes; 192/384/768-byte codes at $d = 768$, 384/768/1,536 at higher dimensions; fp16/fp32 lookup settings; probes 16–512
IVF-RaBitQ	4/8/9 bits per dimension, supported search modes, probes 16–1,024; additional 4,096-list OpenAI indexes
GPU brute force	Exhaustive FP32 squared-L2 search, common host boundary

Dominance and the measured frontier are

$$a \succ b \iff R_a \geq R_b,\ Q_a \geq Q_b,\ (R_a, Q_a) \neq (R_b, Q_b).$$

$$Q_{\text{meas}}(r) = \max\{Q_i : R_i \geq r\}. \tag{3.3}$$

No qualifying measured point means undefined. For adjacent frontier points,

$$\log Q_{\text{int}}(r) = (1-\lambda)\log Q_a + \lambda \log Q_b, \qquad \lambda = \frac{r - R_a}{R_b - R_a}. \tag{3.4}$$

Within 5×10^{-4}, the parser snaps; otherwise it interpolates without extrapolation. Qualifying endpoints remain eligible in the small-batch comparison. Near ties report both estimators; memory is unmatched.

Failed/invalid outputs are excluded; repeat-ID flags alone are not. Isolated reruns replace contaminated builds; skipped Stella batch-1,000 runs add no points (section 3.5.2). Int8 remains included (figure 7.1); ADC/poster plots have separate scopes.

3.5 Experimental Records and Reproducibility

Experiment labels identify distinct measurement families. E1 names the main batch-1,000 frontier sweep; D compares TensorFloat-32 (TF32) and FP32 JHQ arithmetic against archived baselines, while T1 remeasures the baselines in-session. T4 sorts GPU scores on CPU; T5 covers small batches. A1/A2/A3 denote construction repeats, shared-bound instructions and dataset alignment; B records budget decisions; C1 audits representation.

3.5.1 Sources and available records

The project archive retains versions, settings, timings, outputs and selected T1, T4/T5 and A2 inputs. Paired runs share index state. Complete vectors, trained states and historical executables/hashes are not all available. T4 candidate dumps were deleted; retained summaries cannot support an independent resort. Sampling and generalisation limits follow sections 6.2, 6.3 and 8.3.

Table 3.4. Implementation families used by component tests. Each experiment retains its own control; versions are not successive whole-system speedups.

Version	Mechanism / experimental role
v61	Word-load and lane-refinement controls
v62 / v63	Grouped full-key selection / shared bound
v65 / v69	Batch/skew scheduling / radix probe selection
v69 / v71 / v73	Budget calibration / LUT sweep / strict-score export
v69 / v72	OpenAI: TF32 versus FP32 add/query arithmetic on the same trained cache

3.5.2 Timing and calibration conventions

Warm-up reduces E1 call counts at 274 points; initial conversion/validation differs from steady-state timing. The 369 repeat-ID flags include tied reordering; 401 also counts cross-protocol differences. Flags alone exclude no points, and JHQ lacks the same flag.

The campaign sets the 200-start grid explicitly. Fixed arms use one warm-up/three timed calls. Calibration times full searches and $O(Sk^2)$ host comparison, excluding gathering/scan calibration and bypassing graph capture. Warm-up visits every query without passing outputs/GT to agreement. Callers install the returned index with `set_alpha(A[index])`; selected-arm rates reuse fixed-arm timing (section 6.5). For 64 sorted losses, the recorded quantile is

$$L_{95} = L^{\text{sorted}}_{\lfloor 0.95(64-1)\rfloor} = L^{\text{sorted}}_{59} \qquad \text{(zero-based)}.$$

Follow-up sessions have separate states/boundaries unless stated. A2 corroborates the PTX contract (section 5.5); T4's score-export scope is in section 7.4.1. T5 includes null calls/int8 query quantisation, with API/warm-up differences (section 7.2.6).

4 GPU Index Design and Construction

Training/encoding produces bounded-memory, list-ordered GPU arrays [16]; chapter 5 and section 7.7 give search and cost.

4.1 Build and Search Workflow

Train fits models; add encodes/list-orders rows; search uses the completed index. Cache hits skip fitting, not encoding, so cold-build and cached-search costs differ.

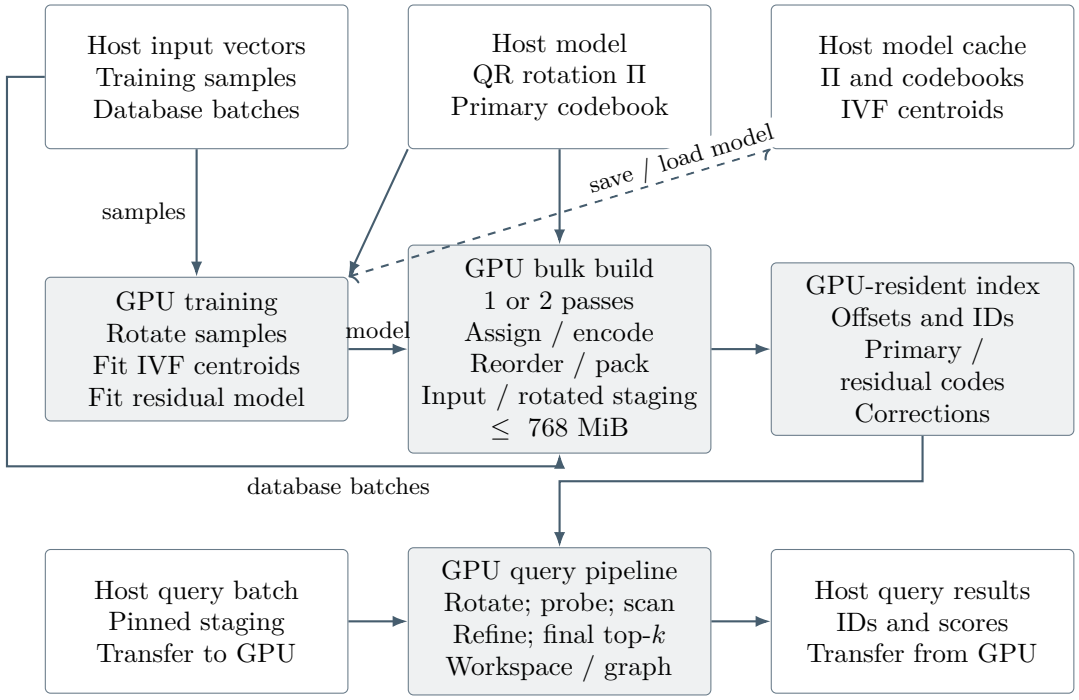


Figure 4.1. Implemented build and query boundaries. Host work includes sampling, orthogonal-matrix construction and analytical primary-codebook generation; the evaluated configuration fits IVF and residual models on the GPU. The cache stores trained state, whereas the completed database representation resides on the device. Dashed arrows: model reuse. The staging bound excludes other scratch.

Hosts prepare samples/rotation/primary codebooks; GPUs rotate, fit IVF/residual models and encode bounded batches into list order.

Models hold rotation, codebooks and centroids; database state holds offsets, IDs, codes and corrections. CUDA Graphs replay recorded GPU operations and reference reused query buffers.

One successful `add` fixes positions. Further adds are rejected; updates require rebuilding. Mutable/transactional publication is unimplemented.

Host IDs/scores and call timing follow equation (5.20) and section 3.3.

4.2 Training, Rotation and Codebook Generation

4.2.1 Transform and sampling

Host QR factorisation of a seeded Gaussian matrix produces square Π:

$$Y_t = X_t \Pi^{\mathsf{T}}, \qquad y = \Pi x, \qquad \|\Pi(q-x)\|_2^2 = \|q-x\|_2^2 \quad \text{when } \Pi^{\mathsf{T}}\Pi = I. \tag{4.1}$$

The geometry contract (C0) checks finite-precision QR/GEMM error; dimension remains d (section 4.6).

Strided primary/IVF training takes $n_t = 39n_{\text{list}}$:

$$n_t = \begin{cases} 159{,}744, & \text{Vogue,} \\ 319{,}488, & \text{arXiv/OpenAI,} \\ 1{,}277{,}952, & \text{BGE/Stella.} \end{cases}$$

This is Faiss's warning threshold, not a quality guarantee [8]. Residual fitting uses 100,000 rows.

An older twelve-cell Vogue/OpenAI-3072 sweep ($B_r = 4/8$, probes 32/128/256) fixed primary/IVF models. With 25 Lloyd iterations, 100K rather than all rows lost at most 0.0018 recall (eleven losses/one tie). Intermediate sizes were non-monotone; this supports bounded training only within that protocol.

4.2.2 Analytical primary codebook

Sample scale and Cartesian levels are

$$\sigma^2 = \frac{1}{n_t d} \sum_{i=0}^{n_t-1} \|x_i\|_2^2, \qquad D_s = d/M, \qquad K = 2^B. \tag{4.2}$$

$$B = D_s = 8 \Longrightarrow \mathcal{C}_m = \{-a, +a\}^8, \quad a = 0.674\sigma.$$

Eight signs fit a byte, enabling $O(d)$ encoding/split LUTs [16] (section 5.2). Arbitrary Lloyd updates may break Cartesian structure.

For fixed sign vector $s \in \{-1, +1\}^d$,

$$\|q' - as\|^2 = \|q'\|^2 + da^2 - 2a\langle q', s\rangle. \tag{4.3}$$

$$a > 0 \Longrightarrow \arg\min_s \|q' - as\|^2 = \arg\max_s \langle q', s\rangle.$$

The order needs $a > 0$, not Gaussian data. Residuals depend on a; zero scale/FP32 ties are separate cases.

4.2.3 Coarse assignment as filter and refine

IVF initialises spaced rotated rows and runs eight iterations using $s_\ell = \|\mu_\ell\|^2 - 2\langle y, \mu_\ell\rangle$. Supported dimensions use int8 filtering, resolving ambiguity in FP32.

Lemma 4.1 (Assignment with certified score intervals)**.**
Let $\widetilde{y}, \widetilde{\mu}_\ell$ be dequantised approximations, $e = y - \widetilde{y}$ and $f_\ell = \mu_\ell - \widetilde{\mu}_\ell$. Keeping centroid norms exact, the approximate score $\widetilde{s}_\ell = \|\mu_\ell\|^2 - 2\langle \widetilde{y}, \widetilde{\mu}_\ell\rangle$ has real-arithmetic error bounded by

$$E_\ell = 2(\|y\|\|f_\ell\| + \|e\|\|\widetilde{\mu}_\ell\|). \tag{4.4}$$

Given valid intervals $[L_\ell, U_\ell]$ containing every target score, discarding only $L_\ell > \min_j U_j$ and exactly resolving the remaining scores preserves the minimum and its specified tie rule.

Proof.
Using $e = y - \widetilde{y}$ and $f_\ell = \mu_\ell - \widetilde{\mu}_\ell$,

$$\langle y, \mu_\ell\rangle - \langle \widetilde{y}, \widetilde{\mu}_\ell\rangle = \langle y, f_\ell\rangle + \langle e, \widetilde{\mu}_\ell\rangle,$$
$$|s_\ell - \widetilde{s}_\ell| \le 2(\|y\|\|f_\ell\| + \|e\|\|\widetilde{\mu}_\ell\|) = E_\ell.$$

For $j^* \in \arg\min_j U_j$,

$$L_\ell > U_{j^*} \Longrightarrow s_\ell \ge L_\ell > U_{j^*} \ge s_{j^*}.$$

The strict inequality excludes ℓ; equality stays eligible. □

FP32 headroom lacks a bitwise certificate without accumulation-error bounds. FP16 differs on 0/14,838 Vogue, 6/31,748 BGE and 3/16,359 Stella tail assignments. Cold logs do not identify precision.

4.2.4 Residual fitting and the iteration budget

Residuals stay $r = \Pi x - \widehat{p}(x)$. Per subspace, GPU sorting/Lloyd fits $K_r = 2^{B_r}$ sorted scalar levels shared across coordinates, capped at 2,000 updates.

Historical 100K training gained 0.0063–0.0076 recall at 25→2,000 updates. Tail-bin/BGE/Stella changes persisted: the cap is not convergence, without contradicting exact Lloyd descent. At 20,000 updates, historical Stella all-row/100K recall was 0.9921/0.9920.

4.2.5 Construction work and observed imbalance

$$W_{\mathrm{QR}} = O(d^3), \qquad W_{\mathrm{rotate}} = O(n_t d^2).$$

Direct coarse assignment costs $O(n_t n_{\mathrm{list}} d)$ per iteration.

$$W_{\mathrm{primary}} = O(MKD_s), \qquad W_{\mathrm{residual\ extraction}} = O(n_r d),$$

followed by sorting and scalar updates. These describe work, rather than predict latency. Synchronised phase diagnostics place about 83% of BGE/Stella training in IVF k-means (about 2.97 s). They use one add buffer and forced synchronisation, separately from the normal overlapped timings in section 7.7.

List-size imbalance enters the scan rule in section 5.5. The individual contributions of eight IVF iterations, spaced seeds and finite sampling to that imbalance were not isolated.

Arithmetic used in the measured implementation. The evaluated implementation (v69) stores FP32 matrices but enables cuBLAS TF32 tensor-core math for $d \geq 1{,}536$. In its compiled faithful branch, a null override falls through to the dimension-based default. This affects training/add/query rotation and centroid GEMMs on both OpenAI datasets; an environment override cannot disable that branch. Thus FP32 storage does not imply full FP32 multiplication. The supplementary FP32 variant (v72) corrects this switch. Its sensitivity experiment reuses the same TF32-trained cache and changes add/query arithmetic, rather than retraining the model in FP32 (section 7.2.3).

4.3 IVF Assignment, IDs and Physical Memory Layout

4.3.1 From database IDs to list-ordered positions

An input row has identifier $i \in \{0, \ldots, N-1\}$. Coarse assignment places its transformed vector in a list according to the computed centroid scores. The mathematical assignment target is

$$a(i) = \operatorname*{arg\,min}_{0 \leq \ell < n_{\mathrm{list}}} \|\Pi x_i - \mu_\ell\|_2^2. \tag{4.5}$$

Its implementation uses the selected numerical assignment path. Encoding the primary and residual representations does not subtract $\mu_{a(i)}$; list membership restricts the future search universe independently of those reconstructions.

The builder sorts assignments together with a sequence of input IDs. Let $\rho(p)$ be the input ID at sorted physical position p and let o_ℓ be the start of list ℓ. The completed arrays satisfy

$$0 = o_0 \leq o_1 \leq \cdots \leq o_{n_{\mathrm{list}}} = N, \qquad p \in [o_\ell, o_{\ell+1}) \Longrightarrow a(\rho(p)) = \ell. \tag{4.6}$$

Lower-bound searches give offsets, with

$$o_\ell = o_{\ell+1} \Longleftrightarrow \text{list } \ell \text{ is empty.}$$

Positions are stable within a snapshot, not across builds. All arrays use the same $\rho(p)$.

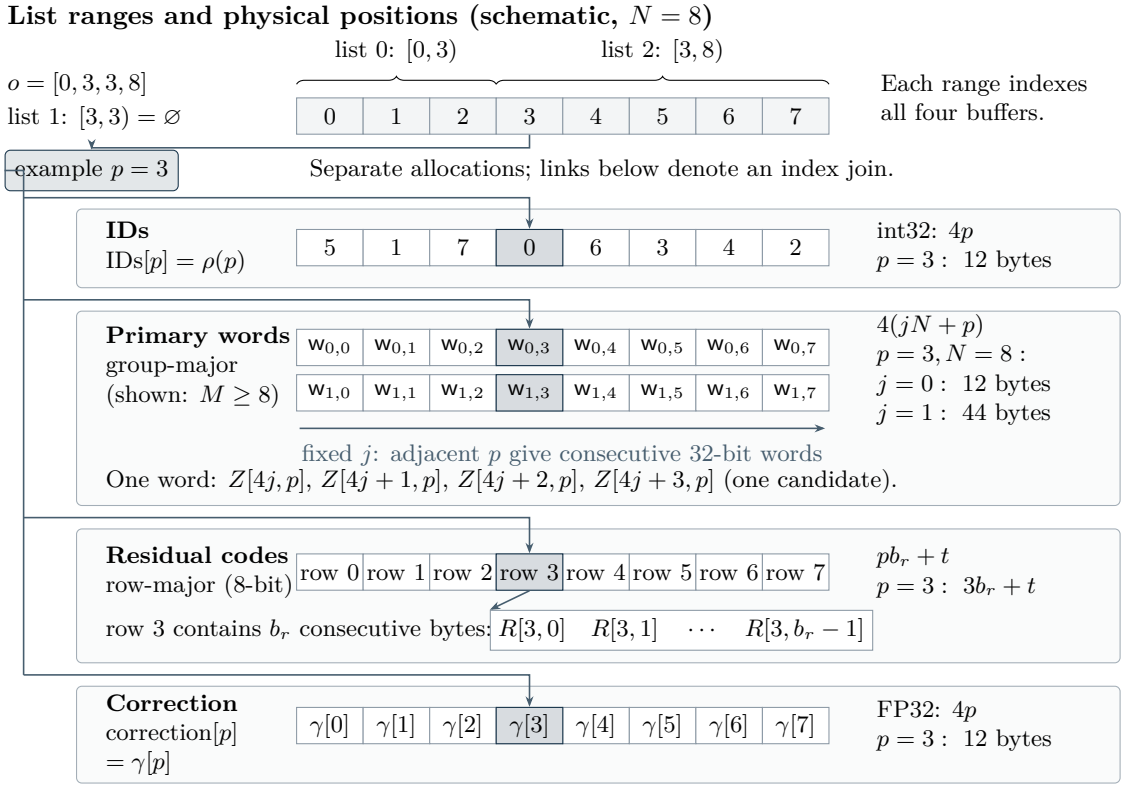


Figure 4.2. Physical GPU index layout (schematic). IVF offsets delimit contiguous physical-position ranges. Independent ID, word-group-major primary, row-major residual and correction arrays share position p. Byte offsets are relative to each allocation; the illustration implies neither interleaved allocation nor measured memory transactions.

IDs map $p \mapsto \rho(p)$. Two-pass add temporarily builds ρ^{-1} for scatter. Primary ties use positions; final ties use database IDs.

4.3.2 Exact addresses and representation widths

For primary byte $Z[m, p]$,

$$\text{offset}_{\text{byte}}(Z[m, p]) = mN + p.$$

Final packing produces

$$\mathsf{w}[j, p] = \sum_{t=0}^{3} 2^{8t} Z[4j + t, p], \qquad \text{addr}_{\text{byte}}(\mathsf{w}[j, p]) = 4(jN + p). \tag{4.7}$$

Here $j = 0, \ldots, M/4 - 1$. word-layout build $\Longrightarrow M \bmod 4 = 0$; other M are rejected. A separate byte-layout compilation retains the byte representation. Each packed array occupies NM bytes, but repacking temporarily holds both old and new copies, costing $2NM$. The one-pass builder temporarily gathers a vector-major $[N, M]$ primary array, transposes it, and then repacks; each obsolete copy is freed. The two-pass builder scatters directly into the subspace-major array before the same final repack. The search-time lane mapping is discussed in Section 5.3.

Residual codes remain row-major, with $b_r = \lceil dB_r/8 \rceil$ bytes per vector. In the eight-bit configuration, coordinate t of row p is at byte offset $pb_r + t$. In the four-bit path it is in byte $pb_r + \lfloor t/2 \rfloor$, with even coordinates in the low nibble and odd coordinates in the high nibble. A

four-byte correction per physical row stores

$$\gamma[p] = 2\langle \widehat{p}(x_{\rho(p)}), \widehat{r}(x_{\rho(p)})\rangle. \tag{4.8}$$

Its value must move with that row's codes during gathering or scattering.

List-order primary scans fetch residuals only for survivors. Raw FP32 reranking would add $4Nd$ device bytes. Here compressed reconstructions are refined; Chapter 7 measures recall.

4.4 Resident Storage, Workspace and Index Lifecycle

4.4.1 Stored bytes and allocated bytes

For byte-sized primary codes,

$$S_{\text{rows}} = N\left(M + \left\lceil \frac{dB_r}{8} \right\rceil + 8\right), \qquad S_{\text{offsets}} = 4(n_{\text{list}} + 1). \tag{4.9}$$

IDs/corrections contribute $8N$ bytes. With $M = d/8$, $B_r = 8$,

$$b_{\text{code}} = d/8 + d = 9d/8 = 1.125d.$$

The code payload exceeds 8-bit scalar quantisation (SQ8) by 12.5%; graph-free need not mean smaller than int8.

Table 4.1. Per-vector bytes. JHQ includes codes, ID and correction; SQ8 is vector codes alone; CAGRA int8 models vector codes plus 32 four-byte neighbours. Offsets, model parameters, workspace and other runtime allocations are excluded. These components are not complete process totals.

d	Primary	Residual	ID + corr.	JHQ	SQ8	CAGRA int8	FP32
768	96	768	8	872	768	896	3,072
1,024	128	1,024	8	1,160	1,024	1,152	4,096
1,536	192	1,536	8	1,736	1,536	1,664	6,144
3,072	384	3,072	8	3,464	3,072	3,200	12,288

At graph degree $d_g = 32$, CAGRA uses $d + 4d_g$ row bytes. Stella JHQ payload (19.20 GiB) is close to int8 CAGRA's 19,656 MiB, under different scopes. α changes residual reads, not stored bytes.

Basic model storage is

$$S_{\text{model}} = 4d^2 + 4MKD_s + 4MK_r + 4n_{\text{list}}(d+1) \quad \text{bytes.}$$

Derived arrays are additional. Workspace holds two query matrices plus centroid scores, LUTs, probes and survivors. Optional residual LUTs add $4b_{\text{cap}}dK_r$ bytes; defaults score directly. Group capacity stays about 93.8 MiB at 128 probes even when unused (equation (5.13)).

Input is caller-owned. Separate staging/order storage is

$$S_{\text{pinned}} = 4n_b a_{\text{in}} d, \qquad S_{\text{order}} = S_{\text{inverse}} = 4N.$$

Assignment scratch ends before gathering (equations (4.10) and (4.13)).

4.4.2 Cache reuse and invalidation

Cache keys omit code version/full-data hashes. Code/data changes need invalidation. Sweeps share states; cold builds refit. Similar recall cannot establish model identity.

Cache hits still pay QR, warm-up, model upload and add. Stella's 6.67 s add includes initial workspace, excludes scan calibration/search warm-up and is not restart time.

α changes need no retraining. Workspace grows before capture; changing batch size, survivors, probes, lanes, groups or bounds triggers recapture. Calibration may bypass capture. Index-owned buffers are reused until release.

4.5 Construction Cost and Tested Scale

4.5.1 Bounded input batches and two-pass construction

Whole-database raw/rotated storage is $4Nd + 4Nd = 8Nd$ bytes. Bounded staging avoids it. For n_b pipeline buffers, configured add rows a_{add} and per-array byte budget S_{stage}, the source stages

$$a_{\text{in}} = \max\left\{1, \min\left(a_{\text{add}}, \left\lfloor \frac{S_{\text{stage}}}{4d} \right\rfloor\right)\right\}, \qquad S_{\text{float}} = 8n_b a_{\text{in}} d. \tag{4.10}$$

$$4d \le S_{\text{stage}}, \quad n_b = 3,\ S_{\text{stage}} = 128\,\text{MiB} \Longrightarrow S_{\text{float}} \le 2n_b S_{\text{stage}} = 768\,\text{MiB}.$$

Assignment/encoder scratch is additional.

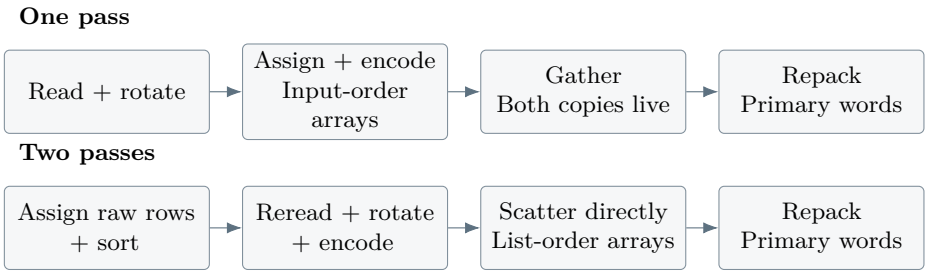


Figure 4.3. Construction paths. One pass briefly holds input-order and gathered encodings; two passes reread/rotate to encode directly into list order. Raw assignment against back-rotated centroids uses orthogonality.

4.5.2 Memory use across construction phases

The one-pass estimate is

$$H_{\text{single}} = N(2b_r + 3M + 20) + 2^{30} \quad \text{bytes}. \tag{4.11}$$

Its ten live arrays sum to

$$3NM + 2Nb_r + 20N = N(3M + 2b_r + 20).$$

The 1 GiB allowance is heuristic, not a process-peak bound.

Final two-pass repack counts IDs, residuals, corrections and two primary copies:

$$S_{\text{repack},2} = N(b_r + 2M + 8). \tag{4.12}$$

Whole-build peak also considers earlier pipeline/inverse scratch, model, sort, context and later workspace allocations.

Table 4.2. Analytical row-array occupancy (GiB), $d = 1{,}024$, $M = 128$, $B_r = 8$. BGE uses one pass; Stella uses two. The one-pass Stella column is counterfactual capacity analysis. Pipeline, model, offsets and runtime costs are excluded from these phase values.

Dataset	Final row payload	One-pass gather	Two-pass repack
BGE	10.90	23.05	12.11
Stella	19.20	40.59	21.32

Table 4.2 quantifies the compressed-copy saving. Stella's cold log selects two passes and prints 41.6 GiB for equation (4.11), against 30.6 GiB available at the tail. The separately sampled resident process totals, including context and query workspace, are 11.93 GiB for BGE and 20.44 GiB for Stella (section 4.4). The supplementary 20 ms device samples observe a 26.08 GiB BGE peak near the start of add, and a 22.58 GiB Stella peak during repack. These are sampled device totals including context; shorter-lived peaks can be missed.

Assignment-product capacity uses an independent configured row count a_{assign}, selected by `JHQ_ASSIGN_BATCH` or the source default: 32,768 for at least 8,192 lists, otherwise 8,192.

$$S_{\text{dots}} = 4 n_b n_{\text{list}} a_{\text{assign}} \quad \text{bytes.} \tag{4.13}$$

Allocation capacity is independent of active/staged rows. At $d = 1536/3072$, defaults both allocate 3 GiB. For BGE/Stella, $3 \times 4 = 12$ GiB assignment scratch precedes gathering; figure 4.4 separates those lifetimes.

$$S_{\text{peak}} = \max_t \sum_{A \text{ live at } t} |A|.$$

Stella's smaller repack plus model/context gives its lower observed peak.

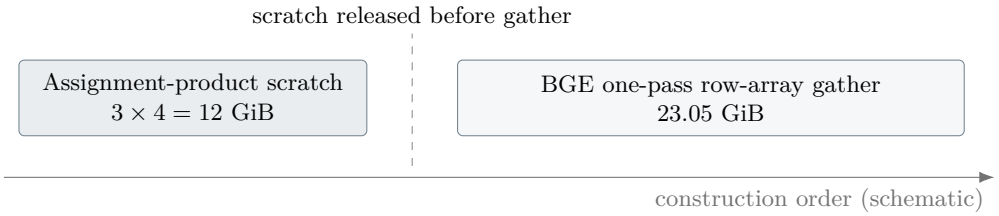


Figure 4.4. Selected construction lifetimes, not total process occupancy. BGE's 12 GiB assignment buffers are freed before its 23.05 GiB row-array gather. These separate terms must not be summed as concurrent memory.

4.6 Index Representation and Validation

A signed-32-bit $n_t d$ overflow at 2^{31} zeroed scale; a version-blind cache reloaded it. Chunked FP64 norms fixed the fault; routing success alone missed it.

4.6.1 Representation consistency checks

Pilot exports cover Vogue, arXiv and OpenAI-3072.

Codebooks passed separability, scalar-level and sorting checks. An FP64 loop checked $\gamma = 2\langle \widehat{p}, \widehat{r} \rangle$ on every exported row; maximum absolute error was 1.28×10^{-7}–2.77×10^{-7}. Sampled GPU-returned top-10 scores have maximum relative errors from 2.38×10^{-7} to 4.85×10^{-5}, using $|s_{\text{GPU}} - s_{64}|/s_{64}$ with the query-norm constant included. These are checks of the exported representation, not re-encoding from the original input vectors.

Across seven workloads and 896 query instances, the pilot additionally checked the proposed residual bounds against FP64 reconstructed distances for 72,800,590 routed candidate instances, with zero reported violations. The same population supports the FP64 composite-score identity check. GPU-returned-score checks cover only top-10 slots and skip IDs absent from the routed map, leaving output membership incompletely checked.

Encoder, primary/residual training and construction sources match the evaluated implementation after namespace normalisation and removal of the export method. This source correspondence covers the shared path, leaving later model identity and large-dataset two-pass construction to separate checks.

4.6.2 Input encoding and data placement checks

The supplementary exporter audit uses the shared construction path on Vogue, arXiv, OpenAI-3072 and Stella. Offsets are monotone with endpoints 0 and N; IDs form a permutation of every database row in all four. FP64 products of the stored rotation give maximum entrywise $\|\Pi\Pi^{\mathsf{T}} - I\|$ errors of 3.96–4.34×10^{-7}. Sampled norm checks use FP64 rotation, rather than the GPU GEMM, and therefore measure the stored matrix's geometry. Table 4.3 summarises sampled recoding.

Table 4.3. Original-input FP64 recoding audit. Flips count coordinate decisions; the final column is the largest residual-boundary distance among mismatches. Structural checks cover all rows; recoding is sampled.

Dataset	Rows	Orth. error	Primary flips	Residual flips	Max margin
Vogue	200,000	4.11e-07	14	5,409	1.12e-07
arXiv	100,000	4.11e-07	1	2,515	7.31e-08
OpenAI-3072	20,000	3.96e-07	5,745	1,646,806	2.67e-05
Stella	100,000	4.34e-07	16	3,064	1.03e-07

At 768/1,024 dimensions, recoding mismatches fall within 1.12×10^{-7} of boundaries. OpenAI-3072 has 2.68% residual mismatches, up to 2.67×10^{-5} from boundaries, consistent with TF32. Conditioning on stored primary digits avoids double-counting flips.

ID-joined builds have identical codes on three smaller databases, corrections within 8.94×10^{-8}, and 1/1/40 Vogue/arXiv/OpenAI assignment differences. Stella checks only two-pass construction. Missing exported centroids leave membership unverified.

4.6.3 Validation coverage and limitations

Table 4.4. Representation contracts after the pilot and supplementary input audit. Checks separate geometry, arithmetic and row placement.

Boundary	Required property	Evidence / remaining check
C0: geometry	Common transform with bounded floating-point isometry error	Stored-matrix orthogonality checked on four datasets; TF32 GPU arithmetic distinguished
C1: codebooks / correction	Cartesian levels, sorted residual codebooks, correction matches decoded codes	Pilot checks completed on three databases; all-row correction errors in section 4.6
C1: input encoding	Stored codes encode the intended original input row	FP64 original-input recoding on four samples; boundary and TF32 discrepancies quantified
C1: placement	Offsets cover rows; IDs form a permutation; all fields follow it	All-row offsets/ID checks; three ID-joined path comparisons; centroid membership unverified
Execution	Valid indexing, synchronisation and score-order contracts	Archived query and sanitizer checks; finite coverage, detailed in section 7.4

Required conditions are

$$M \mid d, \quad D_s \mid B, \quad M \bmod 4 = 0.$$

Require finite inputs/valid counts and sentinels. IDs are signed 32-bit; address products use wider arithmetic.

T4 validates selection (section 7.4.1); centroid membership, empty lists and adversarial tails remain uncertified.

5 Representation-Aware GPU Search

GPU scoring/selection uses JHQ's hierarchy [16]. Chapters 6 and 7 evaluate it and set budgets.

5.1 Query Workflow and Ranking Scores

Queries rotate, route, screen and refine to return k IDs/scores. Figure 5.1 distinguishes the three runtime selection decisions; the ground-truth ordering is an external evaluation object.

Four quantities must be kept separate:

$$\begin{aligned} s_i(q) &= \|\mu_i\|_2^2 - 2\langle q', \mu_i\rangle, && \text{centroid routing score,} \\ D_{\mathrm{P}}(q,x) &= \|q' - \widehat{p}(x)\|_2^2, && \text{primary score,} \\ D_{\mathrm{H}}(q,x) &= \|q' - \widehat{p}(x) - \widehat{r}(x)\|_2^2 + \|q'\|_2^2, && \text{refined ranking score,} \\ D_{\mathrm{raw}}(q,x) &= \|q - x\|_2^2, && \text{raw-space distance.} \end{aligned} \tag{5.1}$$

μ_i is a transformed centroid; $\widehat{p}, \widehat{r}$ are decoded primary/residual vectors. The runtime refined score includes a query-only constant, as derived from the implemented correction in Section 5.6. It preserves the ordering of reconstructed distances for a fixed query, but neither it nor the primary score is the original-vector distance.

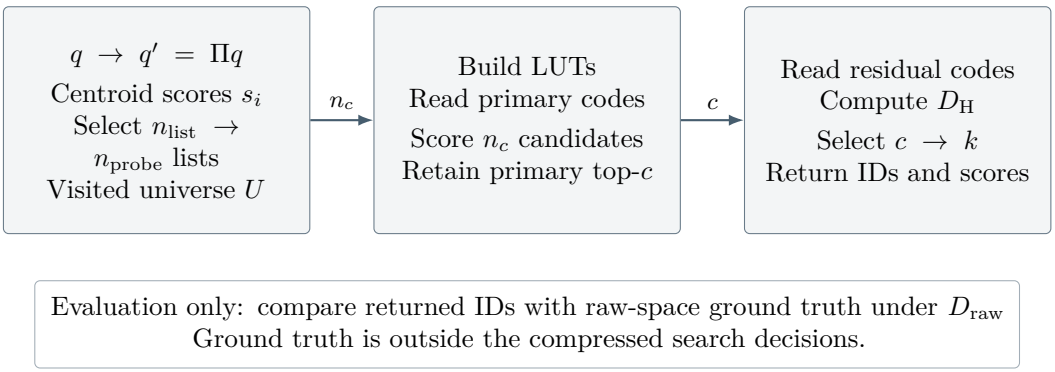


Figure 5.1. Query execution and its separate selection boundaries. Routing selects lists, primary screening selects vector candidates, and final selection ranks refined survivors. Raw-space ground truth is used for evaluation; it is not an input to these search decisions.

Routing defines the visited universe U of n_c candidate occurrences; primary screening retains up to $c = \max\{k, \lceil \alpha k \rceil\}$ for residual scoring. Table 5.1 records each stage's inputs, reads and costs; sections 5.2 to 5.7 develop their implementations.

The ordering and exactness columns of table 5.1 attach each claim to its score and tie rule. Chapter 7 tests survivor sets, ordered outputs and raw-space recall separately.

Table 5.1. Query-stage contracts. $c_v = \min(c, n_c)$ counts valid primary survivors. Exactness refers to the stated computed scores; routing and quantised reconstruction determine raw-space approximation.

Stage	Input → output	Data read	Main work	Order / exactness
Transform	$q \to q'$	Query; transform	Apply Π	Transform (precision in section 7.2.3)
Probe	$n_{\text{list}} \to n_{\text{probe}}$ lists	FP32 centroids; list lengths	Score; radix; rank count	(s_i, i); exact probe selection
LUT	$q' \to$ tables	Primary codebook	$MD_s 2^{8/g_{\text{LUT}}}$ terms	Real-arithmetic factorisation; FP32 rounding
Primary, $g_{\text{scan}} = 0$	$n_c \to c_v$	Packed primary codes; LUT	$O(n_c M)$ scoring; compaction	Exact distance-order selection; arbitrary cutoff ties
Primary, $g_{\text{scan}} \geq 1$	$n_c \to c_v$	Same primary representation	Score; compact; merge	(D_{P}, p); exact under stated premises
Refine	$c_v \to c_v$ scores	Residual codes; correction	$O(c_v d)$	Computed D_{H}; lane-dependent FP32 association
Final	$c_v \to k$	Refined scores; IDs	$O(kc)$ slot scans	$(D_{\text{H}}, \text{ID})$; exact with k finite valid entries

Table 5.1 gives the stage contracts. The E1/budget harness explicitly selects 16 refinement lanes and scan policy -1 (scheduling among 0/2/4/8), versus library defaults of scalar refinement and policy 0. Shared bounds are enabled but used only at $g_{\text{scan}} > 1$; the harness calibrates scheduling with 512 database pseudoqueries, a step the library does not perform automatically. Component overrides and implementation versions are in table 3.4.

5.2 Exact Cartesian LUT Factorisation

Primary ADC uses FP32 q' and stored reconstructions [16, 17]:

$$T_m[z] = \|q'_m - \mathcal{C}_m[z]\|_2^2, \qquad D_{\text{P}}(q, x) = \sum_{m=0}^{M-1} T_m[z_m(x)]. \tag{5.2}$$

The 256-entry table can shrink because each subcodebook is Cartesian.

For $K = 256$, $D_s = 8$, the analytical codebook gives

$$\begin{aligned} a &= \sigma\Phi^{-1}(3/4) \simeq 0.67449\sigma, \qquad \mathcal{C}_m = \{-a, +a\}^8, \\ D_{\text{P}}(q, x) &= \|q'\|_2^2 + da^2 - 2a \sum_{j=0}^{d-1} s_j(x) q'_j. \end{aligned} \tag{5.3}$$

Here Φ is the standard normal CDF; a byte stores eight signs, higher bits first. Thus

$$a > 0 \quad \Longrightarrow \quad \arg\min_x D_{\text{P}}(q, x) = \arg\max_x \sum_j s_j(x) q'_j.$$

Four-/eight-way splits retain the same real-arithmetic score. Learned replacements require compatibility checks.

5.2.1 Factorisation without changing the quantiser

Write $z = 16h + l$, with $h, l \in \{0, \ldots, 15\}$ and $\mathcal{C}_m[z] = (A_m[h], B_m[l])$. Then

$$T_m^{\text{hi}}[h] = \|q'_{m,\text{hi}} - A_m[h]\|_2^2, \qquad T_m^{\text{lo}}[l] = \|q'_{m,\text{lo}} - B_m[l]\|_2^2. \tag{5.4}$$

Coordinate separation gives

$$T_m[z] = T_m^{\text{hi}}[z \gg 4] + T_m^{\text{lo}}[z \,\&\, 15]. \tag{5.5}$$

Storage falls to $16 + 16 = 32$ entries; codes, candidates and real-arithmetic scores stay fixed (figure 5.2).

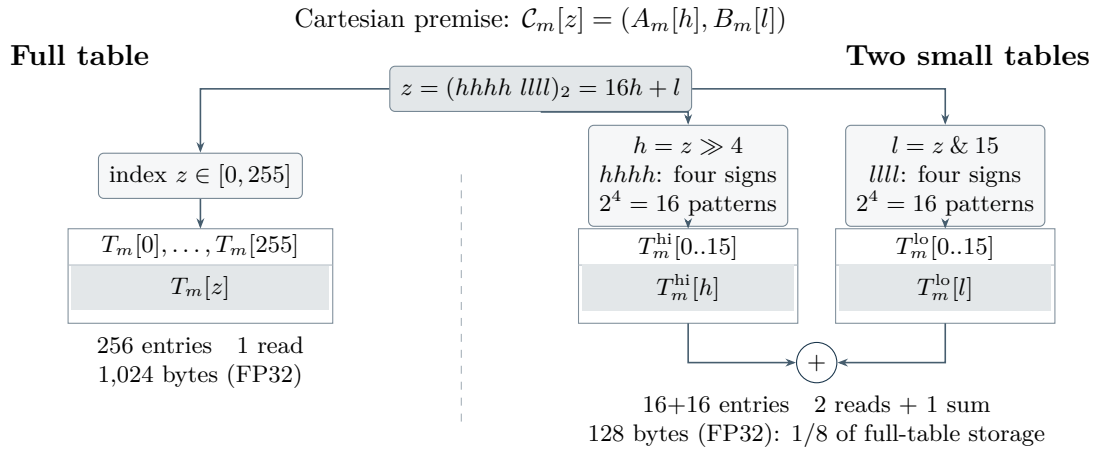


Figure 5.2. Cartesian primary-LUT factorisation (schematic). A byte selects one entry in a 256-entry table, or one high- and one low-nibble entry in two 16-entry tables. Coordinate separation gives the additive identity in real arithmetic; FP32 rounding can change top-c and final IDs. T4's maximum relative score difference across table forms is 4.64×10^{-7}; Vogue has 2–4/256 final-ID differences, arXiv/OpenAI-3072 none (section 7.3). Reads and sum describe the lookup expression.

The builder slices codewords $16h, l$; `verify_split_lut` checks nibble/slice compatibility. An arbitrary learned 256-centroid PQ subcodebook need not satisfy this condition: dividing its code labels into nibbles does not factor its centroid coordinates.

The same-candidate FP32 audit in section 7.3 measures numerical and ranking differences between table forms, including changes after identical primary survivor sets.

5.2.2 Grouping, construction work and resource cost

The table-grouping ablation generalises the split to $g_{\mathrm{LUT}} \in \{1, 2, 4, 8\}$ contiguous coordinate groups whose bit fields match the Cartesian digits. For eight bits and $D_s = 8$, each group has $2^{8/g_{\mathrm{LUT}}}$ entries; the total entries and FP32 storage per query are

$$n_{\mathrm{LUT}}(g_{\mathrm{LUT}}) = g_{\mathrm{LUT}} 2^{8/g_{\mathrm{LUT}}}, \qquad S_{\mathrm{LUT}}(g_{\mathrm{LUT}}) = 4M n_{\mathrm{LUT}}(g_{\mathrm{LUT}}) \text{ bytes.} \tag{5.6}$$

Independent entry construction evaluates $D_s 2^{8/g_{\mathrm{LUT}}}$ coordinate contributions per subspace. Two groups therefore reduce table storage eightfold and coordinate work sixteenfold relative to independently constructing every full-table entry. These are table-level cost ratios, not predicted application speedups.

Table 5.2. Cartesian table trade-offs for an eight-bit, eight-coordinate primary subspace. Combining additions are the algebraic minimum for joining the looked-up values, excluding accumulation across subspaces; they are not measured instruction counts.

g_{LUT}	Entries	FP32 bytes	Lookups	Combining adds	Build terms
1	256	1,024	1	0	2,048
2	32	128	2	1	128
4	16	64	4	3	32
8	16	64	8	7	16

The table is built in global workspace and staged cooperatively when its complete shared-memory footprint fits the queried per-block opt-in limit $S_{\max}$. For buffer capacity C_{buf}, ordinary/grouped scan footprints are

$$S_0 = 8C_{\mathrm{buf}} + 8 + S_{\mathrm{LUT}}(g_{\mathrm{LUT}}), \qquad S_{\mathrm{grp}} = 8C_{\mathrm{buf}} + 24 + S_{\mathrm{LUT}}(g_{\mathrm{LUT}}) \quad \text{bytes.} \tag{5.7}$$

For the evaluated paths (without optional fused-refinement/early-exit scratch),

$$\text{shared lookup} \iff S_{\text{path}} \le S_{\max}.$$

In the table sweep, $c = 1{,}000$, block sizes are 512 or 1,024, and $C_{\text{buf}} = 2{,}048$. Full tables span 96–384 KiB, versus 12–48 KiB for two groups. NVIDIA specifies a 99 KiB per-block limit for compute capability 12.0 [24]. With this limit, the full-table footprint exceeds capacity even at $M = 96$ (114,696 bytes in the query-centric path), whereas the largest two-group footprint is 65,560 bytes in the grouped path. Thus the source-level placement rule sends full tables to global memory and two-group tables to shared memory in these configurations. This placement is inferred from the launch rule and published limit. The measurement records do not include a device-attribute query.

Four and eight groups both store 16 entries per subspace but perform more lookups and combinations. Extraction/dependencies/staging/synchronisation/residency set the trade-off. Each scan group may stage a copy.

The 16 + 16 nibble LUT follows small-table precedents [1, 2, 33]. Here GPU retention/refinement integrate it with JHQ's two-level score; measured trade-offs follow in section 7.3.

5.3 Packed Codes and Candidate-Parallel Memory Access

Subspace-major codes align neighbouring lanes/candidates; splitting reduces LUT storage.

5.3.1 Address mapping and four-byte packing

For list ℓ, row r maps to

$$p = o_\ell + r, \qquad \text{offset}(Z[m,p]) = mN + p.$$

At fixed m, $\Delta\,\text{offset} = \Delta p$. List boundaries can break adjacency.

Pack four subspace bytes per candidate:

$$\mathsf{w}[j,p] = \sum_{t=0}^{3} 2^{8t} Z[4j+t,p], \qquad \text{offset}_{\text{bytes}}(\mathsf{w}[j,p]) = 4(jN+p). \tag{5.8}$$

For $0 \le t < 4$ and eight-bit Z,

$$(\mathsf{w}[j,p] \gg 8t)\ \&\ 255 = Z[4j+t,p].$$

Nibbles index equation (5.5).

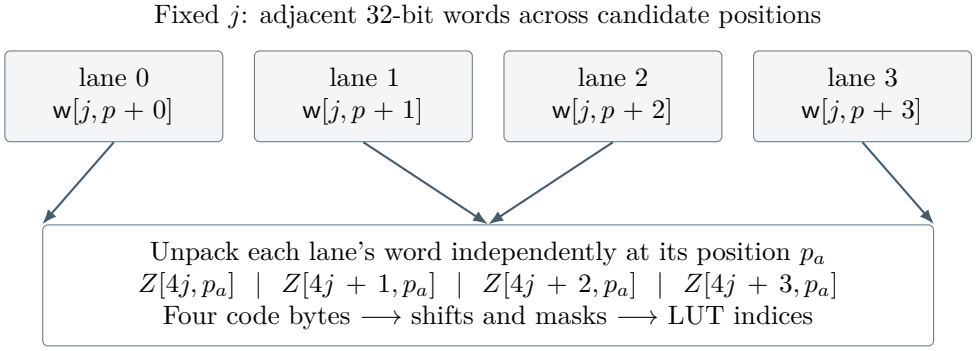


Figure 5.3. Packed candidate-parallel access, showing a subset of warp lanes. At fixed word group j, consecutive positions access adjacent four-byte words. Each word contains four subspace codes of one candidate. Unpacking preserves each code byte; LUT splitting happens afterwards. The diagram illustrates addresses, not a measured number of memory transactions.

Construction maps $[M, N]$ bytes to $[M/4, N]$ words $(4 \mid M)$. Casting would combine candidate bytes. Code-byte costs:

$$R = W = NM, \qquad S_{\text{old+new}} = 2NM.$$

Repacking is outside search timing.

5.3.2 Memory traffic and score consistency

One word load/unpack replaces four byte loads. The logical warp window is $32 \times 4 = 128$ bytes; transactions depend on alignment/active lanes. Payload stays M bytes per candidate. Both arms are subspace-major, so transpose/cache effects are not isolated.

5.4 Exact Global Primary Top-c via Multi-Block Scanning

Groups scan in parallel and merge computed-primary survivors [18].

5.4.1 Candidate partition and deterministic ordering

For a fixed query, concatenate the selected inverted lists in probe order to obtain n_c candidate occurrences. Assume that each visited database row occurs once and that the index stays fixed during search. For $g_{\text{scan}} \in \{1, 2, 4, 8\}$, block (q, j) owns the interval

$$U_j = \left[\left\lfloor \frac{jn_c}{g_{\text{scan}}} \right\rfloor, \left\lfloor \frac{(j+1)n_c}{g_{\text{scan}}} \right\rfloor\right), \qquad j = 0, \ldots, g_{\text{scan}} - 1. \tag{5.9}$$

Here an interval denotes the corresponding candidates, rather than their database IDs. The device computes its endpoints from the routed candidate count. Groups differ in size by at most one candidate; a long list can cross a group boundary. This balances candidate counts, whereas assigning equal numbers of lists need not do so. Each thread finds its first list through the probe-prefix offsets and advances its list cursor while scanning its interval. Residual codes are not read at this stage.

The selection key is

$$K_q(x) = (D_{\text{P}}(q, x), p(x)), \qquad (a, p) \prec (b, r) \iff a < b \text{ or } (a = b \text{ and } p < r), \tag{5.10}$$

where $p(x)$ is the physical row position in the fixed inverted-list storage. It is unique among the query's candidates and independent of the order of atomic appends. It is not the database ID used to break final-result ties. With non-NaN scores, this defines a total order on distinct candidates. Local selection and merging use this order. Partition independence also requires identical computed scores across paths.

5.4.2 Exact local retention and global coverage

$\text{Top}_c(V)$ contains the first $\min(c, |V|)$ keys, where $c = \max\{k, \lceil \alpha k \rceil\}$. Each group keeps up to c, since all global winners may lie there; a c/g_{scan} quota is unsafe.

Lemma 5.1 (Local-buffer retention)**.**
Let V_t be the candidates scored by a block through chunk t, and B_t its valid buffer after any compaction or filtering at that point. Assume fixed complete keys, sufficient append capacity and exact top-c compaction. Let β_t be the minimum of the complete shared bounds observed so far, including the initial top sentinel. Then

$$\mathrm{Top}_c(V_t) \cap \{x : K_q(x) \preceq \beta_t\} \subseteq B_t. \tag{5.11}$$

With sharing disabled, the bound is the sentinel and the buffer contains the whole prefix top-c.

Proof.
The empty prefix is immediate. Put

$$E_t = \mathrm{Top}_c(V_t) \cap \{x : K_q(x) \preceq \beta_t\}.$$

On extending V_t to V_{t+1}, $\beta_{t+1} \preceq \beta_t$ and

$$\mathrm{Top}_c(V_{t+1}) \cap V_t \subseteq \mathrm{Top}_c(V_t).$$

Consequently,

$$x \in E_{t+1} \cap V_t \Rightarrow x \in E_t \subseteq B_t.$$

A new key rejected locally has c predecessors, hence rank $> c$. Capacity admits the remaining eligible arrivals. For any eligible winner x,

$$\#\{y \in B : K_q(y) \prec K_q(x)\} \leq \#\{y \in V_{t+1} : K_q(y) \prec K_q(x)\} < c,$$

so exact compaction keeps x. Filtering removes $K > \beta_{\text{seen}}$; the running minimum covers stale reads. These steps preserve the invariant inductively. □

Proposition 5.1 (Coverage and truncated merging)**.**
For fixed keys under a common total order, let U be the visited universe and V any retained subset. If $\mathrm{Top}_c(U) \subseteq V \subseteq U$, then $\mathrm{Top}_c(V) = \mathrm{Top}_c(U)$. Pairwise sorted merging with truncation to c preserves this result. In particular, for disjoint groups,

$$\mathrm{Top}_c\left(\bigcup_j U_j\right) = \mathrm{Top}_c\left(\bigcup_j \mathrm{Top}_c(U_j)\right). \tag{5.12}$$

The statement includes universes smaller than c.

Proof.
Every global winner is in V, and every other member of V follows the global winners. Selection from V therefore recovers them. For the independent-group special case, let x belong to the global top-c and to group U_j. Fewer than c candidates precede x globally, so fewer than c precede it within U_j. Consequently $x \in \mathrm{Top}_c(U_j)$, and the local union contains every global winner. Selecting its first c therefore recovers precisely the global winners in the same order. If the whole universe has fewer than c members, every member is retained.

For merging, the first c of two sorted lists are their union's top-c. A discarded candidate already has at least c predecessors within that union and cannot enter the top-c of any larger union. Induction over the merge tree establishes the claim. □

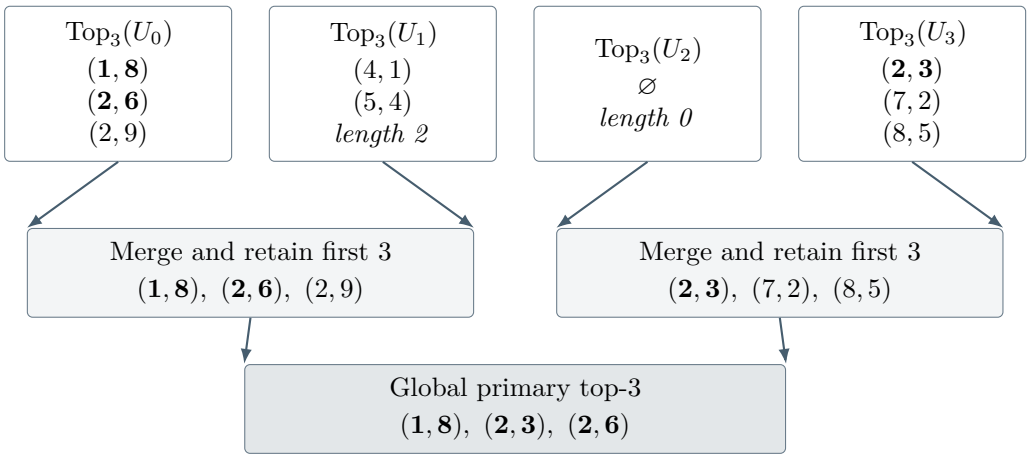


Figure 5.4. Local coverage and truncated pairwise merging, illustrated with $c = 3$ and $g_{\text{scan}} = 4$. Entries are illustrative (D_{P}, p) keys; bold ones are global winners. Short/empty groups contribute their valid prefixes. Merges keep at most c in the same order.

The CUDA implementation maintains a shared-memory buffer of C_{buf} key pairs, where C_{buf} is a power of two and $C_{\text{buf}} \geq c + T_{\text{blk}}$ for a scan block of T_{blk} threads. Bitonic compaction keeps up to c valid keys and sets the threshold to the last key once c exist. Before that point the threshold is a sentinel above every valid key. Between compactions, admission requires $K \prec \lambda_j$. The threshold has c witnesses, so remains an upper bound on the prefix cth key as candidates arrive.

Capacity follows from

$$|B| + |\Delta V| \leq c + T_{\text{blk}} \leq C_{\text{buf}}.$$

This supplies lemma 5.1's append premise. With propositions 5.1 and 5.2, filtering/merging retain global winners.

Explicit scores need $O(n_c)$ writes/storage. Fused buffering emits $O(g_{\text{scan}}c)$ pairs; register selection is another option [18].

5.4.3 Merge implementation, storage and boundary cases

At $g_{\text{scan}} > 1$, a 256-thread block merges each query over $\log_2 g_{\text{scan}}$ rounds. Each pair emits $\min(c, a + b)$ entries using Merge Path slices [14]. All comparisons use equation (5.10); block barriers separate rounds. Figure 5.4 illustrates the truncation invariant. The tree emits at most $c(g_{\text{scan}} - 1)$ key pairs across its internal nodes, in addition to the partition searches and synchronisation work.

Two alternating pair workspaces have shapes $[b_{\text{cap}}, g_{\text{scan}}, c]$ and $[b_{\text{cap}}, g_{\text{scan}}/2, c]$:

$$S_{\text{pairs}} = 8 b_{\text{cap}} c \left(g_{\text{scan}} + \frac{g_{\text{scan}}}{2} \right) \quad \text{bytes}, \qquad g_{\text{scan}} > 1. \tag{5.13}$$

Counts/bounds add $4 b_{\text{cap}} g_{\text{scan}}$ and, when provisioned, $8 b_{\text{cap}}$ bytes; survivor/other workspace is additional. Allocations persist after growth before capture. Merge shared memory holds lengths, not all keys.

Counts delimit valid prefixes:

$$K_{\text{sort pad}} = (+\infty, \text{INT_MAX}), \qquad K_{\text{output pad}} = (+\infty, -1).$$

Valid positions are below INT_MAX; empty groups have zero count. Negative positions are skipped/excluded. $g_{\text{scan}} = 1$ writes directly.

One block ($g_{\text{scan}} = 0$) is distance-exact with free cutoff ties. Lossy `K_LOCAL=4` requires `JHQ_EXACT_TOPCK=0`, disabled in paper-faithful builds.

For nonempty V, put $h = \min(c, |V|)$, with hth score d_h. Distance-only compaction yields

$$\{x : d_x < d_h\} \subseteq B \subseteq \{x : d_x \leq d_h\}, \qquad |B| = h.$$

Each rejection has c no-larger witnesses; tied IDs remain free. T4 checks this contract and grouped full-key order by CPU sorting GPU-produced scores (section 7.4.1).

5.5 Shared Complete-Key Thresholds and Execution Scheduling

Local top-c can emit $g_{\text{scan}}c$ intermediate entries for c winners. Shared bounds reduce this maintenance. The scheduler then chooses how much scan parallelism to expose. Both operate at a fixed refinement budget; choosing that budget is the separate problem in Chapter 6.

5.5.1 A complete-key upper bound with explicit witnesses

Let $K_{(c)}$ be the global cth key when $|U| \geq c$. If a group holds c distinct, fully scored candidates, the largest key among those witnesses, τ_j, satisfies $K_{(c)} \preceq \tau_j$. There are already c candidates no later than τ_j in the global universe. The shared bound is initialised to a top sentinel and updated by

$$\tau \leftarrow \min_{\prec}(\tau, \tau_j). \tag{5.14}$$

No bound is published from fewer than c valid candidates. If the entire universe has fewer than c, no group can publish such a bound and all valid candidates remain eligible.

Proposition 5.2 (Safe rejection by a witnessed bound)**.**
Assume fixed keys, publications backed by c distinct candidates, and reads that return either the initial sentinel or one complete published key. Rejecting a candidate only when $K_q(x) \succ \tau$ cannot remove a global top-c candidate, irrespective of publication timing.

Proof.
A publication has a set W_τ of c distinct witnesses:

$$\max_{x \in W_\tau} K_q(x) \preceq \tau \quad \Longrightarrow \quad K_{(c)} \preceq \tau.$$

Sentinel reads and minima of published keys obey the same inequality. Thus

$$x \in \text{Top}_c(U) \implies K_q(x) \preceq K_{(c)} \preceq \tau_{\text{seen}} \implies x \text{ is not rejected.}$$

Stale reads only loosen the bound. Publications made after earlier safe rejections remain valid: any c retained distinct candidates still supply c witnesses, whether or not they are the best c of their group. □

Shared filtering retains global coverage (proposition 5.1). Admission is

$$K_q(x) \prec \lambda_j \quad \text{and} \quad K_q(x) \preceq \tau_{\text{seen}}, \tag{5.15}$$

Here λ_j is the local compaction threshold and τ_{seen} the last shared key read. The shared comparison retains equality.

5.5.2 Publication, filtering and implementation limits

For nonnegative FP32 scores, canonical positive zero and valid nonnegative positions,

$$\kappa(D, p) = 2^{32}\,\text{uintbits}_{32}(D) + p. \tag{5.16}$$

On this non-NaN domain, monotone float bits give

$$(D_1, p_1) \prec (D_2, p_2) \iff \kappa(D_1, p_1) < \kappa(D_2, p_2).$$

Finite keys precede infinity and the all-one sentinel. Invalid positions cannot publish.

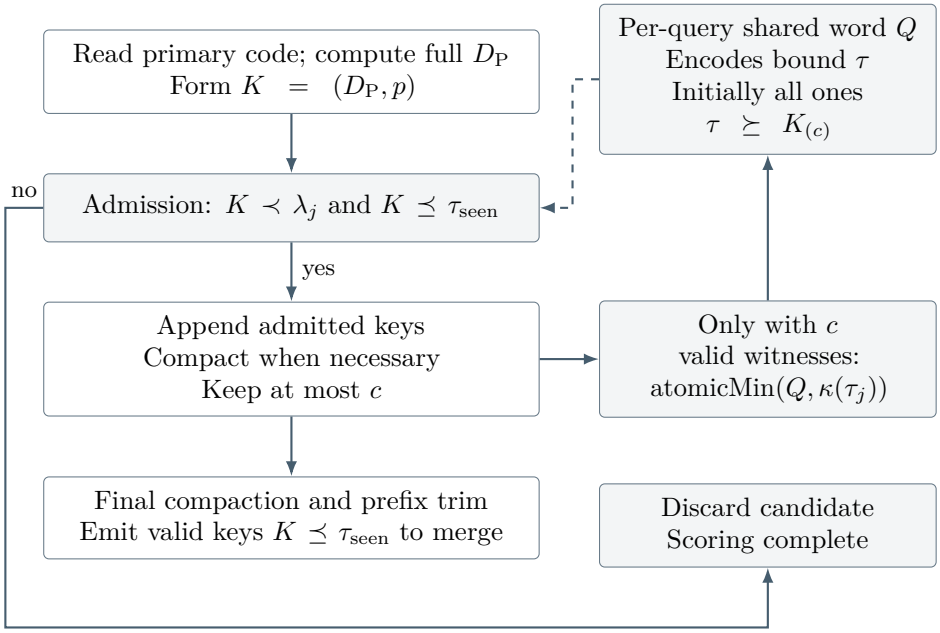


Figure 5.5. Shared complete-key filtering for one scan group. A candidate's full primary score is computed before admission. A valid c-candidate witness set can tighten the per-query bound; a stale but complete bound only weakens filtering. Final prefix trimming reduces the valid merge input, while the allocated workspace capacity remains unchanged. The whole-key-read premise is qualified in the text.

A thread polls each chunk. After compaction, c valid keys permit 64-bit `atomicMin`. Final publication/search trims the prefix; exact merge handles stale extras. In-stream reset precedes scanning, then merging; groups never wait.

The whole-key premise uses aligned eight-byte strong PTX `ld.volatile.global.u64`/`atom.global.min.u64` [26], beyond C++ `volatile` [25]. CUDA 13.0.88 `ptxas`, `sm_120` corroborates it; driver-JIT SASS was not captured.

Filtering follows full primary scoring (figure 5.5): maintenance savings compete with polling/atomics (section 7.4).

5.5.3 Choosing parallelism without changing the budget

bg_{scan} blocks add parallelism and staging/selection/merge costs. Smaller quotas require a certificate or fallback. Concentrated pilot survivors motivated witnessed bounds (section 7.4).

Define

$$T_{\text{blk}} = \text{scan threads/block}, \quad H = n_{\text{SM}} T_{\text{resident/SM}}, \quad v = \frac{\text{sd}(n_c)}{\text{mean}(n_c)}.$$

Its rule is

$$g_{\text{fill}} = \max(\{1\} \cup \{h \in \{2,4,8\} : bhT_{\text{blk}} \le 0.6H\}), \tag{5.17}$$

$$g_{\text{skew}} = \begin{cases} 4, & v \ge 0.3, \\ 1, & v < 0.3, \end{cases} \qquad g_{\text{chosen}} = \max(g_{\text{fill}}, g_{\text{skew}}). \tag{5.18}$$

The skew rule may override the fill limit. In the evaluation harness, v is calibrated using 512 database vectors as pseudoqueries and a fixed routing configuration; it uses no ground-truth labels. This is distinct from Chapter 6's sampled output-agreement calibration. Once calibrated, scheduling uses the batch size and stored statistic without an additional per-search host readback. The constants were fitted on four pilot configurations and then evaluated on further configurations. The one-time pseudoquery calibration is excluded from the reported steady-state search times. Observed startup ranges, separate from refinement-budget calibration

(section 3.5.2), are

$$T_{\text{gather}} \in [0.81, 7.04]\,\text{ms}, \quad T_{\text{first}} \in [6.00, 27.77]\,\text{ms}, \quad \text{median}(T_{\text{repeat}}) \in [2.72, 18.13]\,\text{ms}.$$

Choosing one group dispatches $g_{\text{scan}} = 0$ at fixed index/probes/budget. Its distance-only ties need a common-order check for identical survivors. Integrated gains/regret are in section 7.6.

5.5.4 Observed execution paths

On the 170-SM RTX 5090 (48 warps/SM) [24],

$$1000 \cdot 2 \cdot 512 = 1{,}024{,}000 > 0.6 \cdot 170 \cdot 48 \cdot 32 = 156{,}672.$$

Fill therefore chooses one group; skew can override to four.

In E1, 9/38 records cross $v = 0.3$, all Vogue (0.338–0.622). Its 27 cells use four groups; 105 arXiv/OpenAI cells use $g_{\text{scan}} = 0$. Their gains use LUTs, packing, lanes and probe selection, without grouped retention/bounds.

Vogue: 3–15 pure-tie differences/1,000 queries, zero non-tie differences, identical final outputs. At 128 probes the tie range is 7–11.

Observed cell counts are

	$G = 0$	$G = 2$	$G = 4$
held-out	10	0	2
$S = 128$	2	8	2

Eight of twelve switch paths. Benefits depend on load size/skew, without a fixed batch-256 cutoff.

The heuristic omits residency/compaction/merge costs; pseudoqueries can age. Worst measured regret is 1.217 at batch 256. Budgets stay fixed; section 5.6 handles refinement.

5.6 Lane-Cooperative Residual Scoring and Final Top-k

Primary screening changes coordinate work from $O(n_c d)$ to $O(cd)$. Lanes cooperate at the same budget.

5.6.1 Reconstructed scoring and the query-only constant

For $\gamma = 2\langle \widehat{p}, \widehat{r} \rangle$, expand in real arithmetic:

$$\begin{aligned} D_{\text{H}}(q, x) &= D_{\text{P}}(q, x) + \|q' - \widehat{r}(x)\|_2^2 + \gamma(x) && (5.19) \\ &= 2\|q'\|_2^2 + \|\widehat{p} + \widehat{r}\|_2^2 - 2\langle q', \widehat{p} + \widehat{r} \rangle \\ &= \|q' - \widehat{p}(x) - \widehat{r}(x)\|_2^2 + \|q'\|_2^2. && (5.20) \end{aligned}$$

The query constant preserves reconstructed-distance order [16].

Scalar scoring sums squared residual differences, then adds primary score/correction. Queries and fitting codebooks use shared memory.

5.6.2 Mapping lanes to words and reducing partial sums

For $\ell_{\text{ref}} = L \in \{1, 8, 16, 32\}$,

$$d_{\text{word}} = 32/B_R, \qquad \text{words of lane } a: \; a, a + L, a + 2L, \ldots.$$

A word holds four eight-bit or eight four-bit codes. Butterfly shuffles sum lane partials; lane zero writes the score.

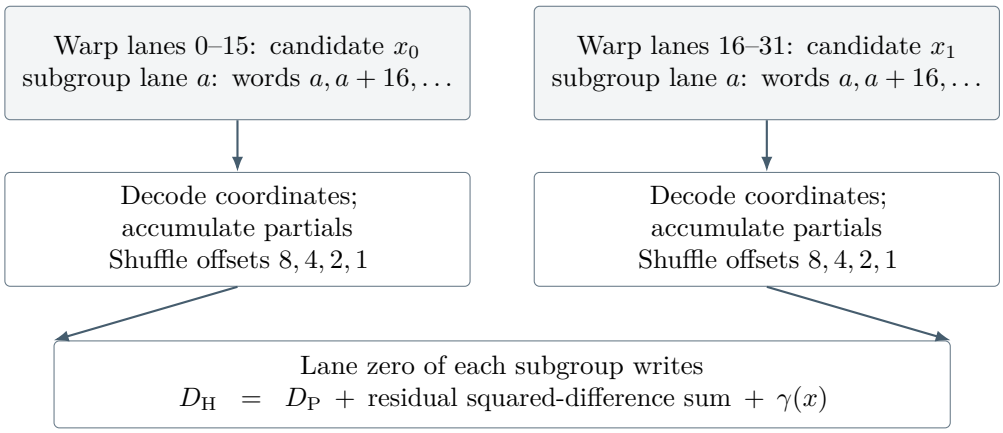


Figure 5.6. Sixteen-lane residual refinement. A warp handles two candidates; lane a within a candidate reads residual words $a, a+16, \ldots$. Each eight-bit-code word decodes four coordinates. Subgroups sum partials, then add primary score/correction. Candidate positions may remain scattered.

Word-aligned rows and $d_{\mathrm{word}} \mid D_s$ keep words within codebooks. All lanes shuffle; invalid slots contribute zero, then output infinity. $L = 16$ gives two candidates/warp.

L0 loads bytes; L1 loads words with the same one-lane accumulation. L8/L16/L32 trade shorter walks for shuffles/fewer concurrent candidates. FP32 association and locality matter.

5.6.3 Final selection and the numerical boundary

For at least k finite valid survivors with unique IDs, each round computes

$$i^* = \arg\min_{i<c}(D_{H,i}, \mathrm{ID}_i), \quad d^* = D_{H,i^*}, \quad D_{H,i^*} \leftarrow +\infty$$

using all slots and emits (ID_{i^*}, d^*): exact retained-pool top-k, without thread quotas. Underfull pools may repeat the smallest valid ID at infinity; all-invalid pools return invalid IDs. Tests meet the finite-entry premise.

Primary ties use positions, final ties IDs. Exclusions/quantisation remain; lane FP32 association may move near ties.

5.7 IVF Probe Selection

Probe selection precedes primary scanning and selects inverted lists, not database-vector survivors. For IVF centroid μ_i, the selector orders the FP32-valued computed routing scores

$$s_i = \|\mu_i\|_2^2 - 2\langle q', \mu_i\rangle, \qquad (s_i, i) \text{ ordered lexicographically.} \tag{5.21}$$

Omitting the common query norm preserves centroid ordering. In contrast to primary squared-distance tables, these scores can be negative. The output is

$$n_{\mathrm{sel}} = \min(n_{\mathrm{probe}}, n_{\mathrm{list}}), \qquad I = \mathrm{Top}_{n_{\mathrm{sel}}}^{(s_i,i)}\{0, \ldots, n_{\mathrm{list}} - 1\},$$

with list-length prefix offsets.

OpenAI v69 uses TF32 rotation/centroid GEMMs with FP32 accumulation. Probe exactness concerns their computed scores; FP32 `fma` retains earlier rounding (section 7.2.3).

5.7.1 Radix selection followed by deterministic ordering

One block makes four MSB-first radix passes. For prefix-restricted histogram H_t and one-based residual rank r_t ($r_0 = n_{\mathrm{sel}}$),

$$b_t = \min\Big\{b : \sum_{a\le b} H_t[a] \ge r_t\Big\}, \qquad r_{t+1} = r_t - \sum_{a<b_t} H_t[a].$$

The chosen bin extends the prefix. Four bytes fix the cutoff/equal-key count.

Canonicalise signed zeros to positive zero, then map bits u by

$$F(u) = \begin{cases} \sim u, & \text{if the sign bit is set,} \\ u \mid 2^{31}, & \text{otherwise.} \end{cases} \tag{5.22}$$

For non-NaN scores, monotonicity follows by cases:

$$\begin{aligned} &D_1 < D_2 < 0 \Rightarrow u_1 > u_2 \Rightarrow \sim u_1 < \sim u_2, \\ &0 \le D_1 < D_2 \Rightarrow u_1 < u_2 \Rightarrow (u_1 \mid 2^{31}) < (u_2 \mid 2^{31}), \\ &D_1 < 0 \le D_2 \Rightarrow F(u_1) < 2^{31} \le F(u_2). \end{aligned}$$

Infinities/zeros are covered, unlike the nonnegative-only equation (5.16). Repeated `fma` fixes recomputed keys.

Histogram storage is

$$256 \text{ bins} \times 32 \text{ lane replicas} \times 4 \text{ bytes} = 32\,\text{KiB}.$$

Replication reduces same-warp collisions, leaving inter-warp contention/reduction cost.

Let h be the cutoff key. Collection forms

$$A_< = \{i : F(u_i) < h\}, \quad A_= = \{i : F(u_i) = h\}, \quad m = n_{\text{sel}} - |A_<|.$$

Take $A_<$ plus the m smallest IDs in $A_=$, using ordered ballots/prefix counts rather than atomic arrival. For the resulting set A,

$$\text{rank}(i) = |\{j \in A : (s_j, j) \prec (s_i, i)\}|, \qquad \text{out}[\text{rank}(i)] = i.$$

Selected lengths produce prefix offsets. For excess probes,

$$i_j = i_{n_{\text{sel}}-1}, \quad L_j = 0 \quad (j \ge n_{\text{sel}}), \qquad \omega_{j+1} = \omega_j + L_j.$$

Zero lengths prevent rescans. Require $n_{\text{list}}, n_{\text{probe}} > 0$.

5.7.2 Costs and separate correctness contracts

Histogram/collection reread scores; ranking is $O(n_{\text{sel}}^2)$ with growing shared memory. Small selections still pay fixed histogram storage; this is deterministic integration, not uniform linear cost.

Tests cover ties/zeros/infinities/extremes/excess probes (section 7.4).

FP16 approximation and cross-process checks are separate (section 7.4).

6 Workload-Level Budget Calibration

Unlabelled agreement with a larger-budget reference selects one multiplier for later batches. Reuse must repay calibration at the required quality.

6.1 Refinement Budget and Search Cost

For routed $U(q)$, the refinement multiplier sets [16]

$$c(\alpha) = \max\{k, \lceil \alpha k \rceil\}, \qquad C_\alpha(q) = \mathrm{Top}^{\mathrm{primary}}_{\min\{c(\alpha),|U(q)|\}} U(q). \tag{6.1}$$

Analysis requires $|C_\alpha| \geq k$; underfull slots are sentinels. Budgets change retention, probes U.

At $k = 10$, the grid retains 20–2,000 survivors without retraining. Lower budgets save reads, not residual storage or previously reserved workspace.

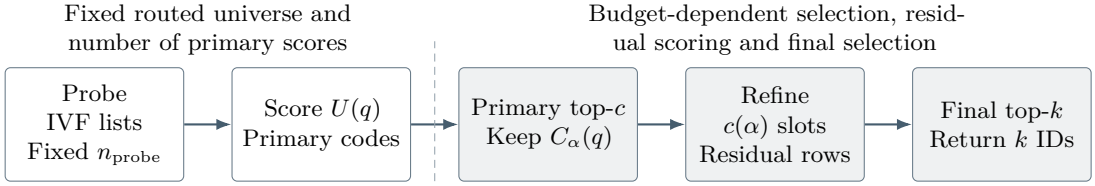


Figure 6.1. The refinement budget changes work after primary-score production. With a fixed query, index and probe count, it leaves the routed candidate universe unchanged. The highlighted stages depend on survivor capacity; this is a logical work decomposition, not a measured latency breakdown. The scan kernel also performs budget-dependent admission and compaction. Padding applies when fewer than c candidates exist.

At fixed batch/routing, write

$$T(\alpha) = T_{\mathrm{base}} + T_{\mathrm{select}}(c(\alpha)) + T_{\mathrm{refine}}(c(\alpha)) + T_{\mathrm{final}}(c(\alpha)). \tag{6.2}$$

T_{base} covers transfers/rotation/routing/primary arithmetic (figure 6.1). Selection/resource transitions make latency non-monotone; the rule minimises accepted budget (section 7.5.5).

Trials reuse the index but rerun full searches, including rotation/probing/primary scoring; calibration costs more than residual work alone.

Position among adaptive ANN methods. α serves the candidate-expansion role of Faiss's `k_factor` and cuVS refinement [7, 10]. Alternatives use GT tuning [9], learned per-query stopping [19] or predicted recall [6]. Here unlabelled agreement chooses one reusable workload budget. Scores still use reconstructed vectors, unlike the cuVS source-vector refinement example.

6.2 Output Agreement and Budget Selection

Let $\mathcal{R}_\alpha(q)$ be the set of k valid, unique database IDs returned at budget α on a fixed index. Define $\mathcal{R}_{\mathrm{ref}}(q) = \mathcal{R}_{\alpha_{\mathrm{ref}}}(q)$. The descending grid is

$$A = (200, 100, 64, 32, 16, 8, 4, 2), \qquad \alpha_{\mathrm{ref}} = A[0] = 200. \tag{6.3}$$

For S calibration queries $\mathcal{Q}_S$, define

$$\begin{aligned} D_S(\alpha) &= \sum_{q \in \mathcal{Q}_S} \left(k - |\mathcal{R}_\alpha(q) \cap \mathcal{R}_{\mathrm{ref}}(q)|\right) \\ &= \frac{1}{2} \sum_{q \in \mathcal{Q}_S} |\mathcal{R}_\alpha(q) \triangle \mathcal{R}_{\mathrm{ref}}(q)|. \end{aligned} \tag{6.4}$$

Accept iff $D_S(\alpha) \le \epsilon = 1$: at most one missing reference ID, ignoring order/distances. The finite-grid objective is

$$\widehat{\alpha} = \min\{\alpha \in A : D_S(\alpha) \le \epsilon\}. \tag{6.5}$$

The reference is a fallback. Section 6.3 gives the conditions for exact bisection.

At $k = 10$, $\epsilon = 1$ gives

S	32	64	128	500
$\epsilon/(Sk)$	0.003125	0.0015625	0.00078125	0.0002

Larger samples change coverage and strictness. Disjoint held-out queries evaluate draws from one finite pool (section 7.5).

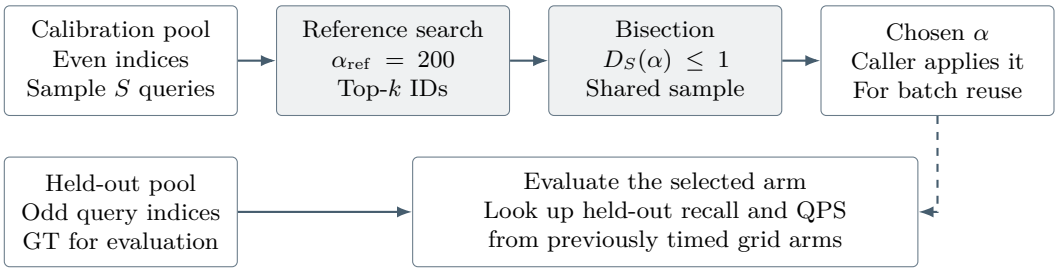


Figure 6.2. Workload-level calibration and its evaluation. Budget decisions use reference IDs; GT assesses held-out quality. Selected-arm timing comes from fixed-arm measurements. Implementation boundaries are in section 3.5.2.

6.2.1 What agreement controls

Under Proposition 6.1, disagreement counts reference answers excluded by primary screening:

$$D_S(\alpha) = \sum_{q \in \mathcal{Q}_S} |\mathcal{R}_{\text{ref}}(q) \setminus C_\alpha(q)|. \tag{6.6}$$

For k-element outputs and GT set $\mathcal{T}_k$, set difference gives

$$|\mathcal{R}_{\text{ref}} \cap \mathcal{T}_k| - |\mathcal{R}_\alpha \cap \mathcal{T}_k| \le |\mathcal{R}_{\text{ref}} \setminus \mathcal{R}_\alpha|.$$

Summing and dividing by Sk yields the relative bound. On the same sample,

$$\begin{aligned} r_{\mathcal{Q}_S}(\alpha) &\ge r_{\mathcal{Q}_S}(\text{ref}) - \frac{D_S(\alpha)}{Sk}, \\ D_S(\alpha) \le \epsilon \implies r_{\mathcal{Q}_S}(\alpha) &\ge r_{\mathcal{Q}_S}(\text{ref}) - \frac{\epsilon}{Sk}. \end{aligned} \tag{6.7}$$

Scope. The bound is relative to the reference on this sample. GT evaluation measures the reference's remaining error (section 7.8.2).

6.2.2 Sampling resolution and ordered testing

Let H_S count queries with any missing reference ID. Since $H_S \le D_S$, acceptance implies $H_S \le 1$. For a prespecified budget and i.i.d. queries, the one-event one-sided 95% binomial limit solves [22]

$$(1-u)^S + Su(1-u)^{S-1} = 0.05. \tag{6.8}$$

The model-dependent resolutions are

	$S = 128$	$S = 500$
i.i.d. query events	0.03652	0.009452
i.i.d. homogeneous result slots	0.003701	0.0009484

The stronger slot model gives approximately $4.74/(Sk)$. Query i.i.d. does not imply slot independence. Neither model certifies fixed-pool draws.

Monotonicity permits fixed-sequence control only with valid per-arm tests [3]. Our one-slot threshold measures agreement, not certified GT risk.

A finite-pool extension. Assume fixed per-query $X_q(\alpha)$, independent of subset/position within the execution regime. For pool size N_{cal} and counts $n_j = |\{q : X_q = j\}|$, uniform sampling without replacement accepts only all-zero samples or one single-count query [23]:

$$\Pr\{D_S(\alpha) \le 1\} = \frac{\binom{n_0}{S} + n_1\binom{n_0}{S-1}}{\binom{N_{\text{cal}}}{S}}. \tag{6.9}$$

Out-of-range binomial coefficients are zero. Predicting GT risk also needs the grid acceptance structure and held-out fixed-arm quality. Recorded subset sums check this empirically (section 7.5.5).

6.3 Budget Search and Monotonicity Conditions

6.3.1 Bisection and complete enumeration

Write $J = |A|$. The implementation computes the sample reference once, then searches for the largest acceptable index in the descending grid:

1. Set $l = 0$, $h = J - 1$. Zero is the fallback.
2. While $l < h$, test $j = \lfloor (l + h + 1)/2 \rfloor$ by running $A[j]$ and computing $D_S(A[j])$.
3. If $D_S(A[j]) \le \epsilon$, set $l = j$; otherwise set $h = j - 1$.
4. Return $A[l]$. Apply it before reuse.

The eight-point grid needs at most three complete candidate searches in addition to the reference. Section 3.5.2 specifies invocation/timing.

Enumeration reruns reference plus seven arms on the same sample. It handles acceptance holes; bisection finds its minimum only for prefix acceptance. Both timers include the reference.

6.3.2 A sufficient condition for monotonicity

Proposition 6.1 (Nested survivors preserve reference membership)**.**
For one query, assume fixed routing and survivor sets $C_\alpha \subseteq C_\beta \subseteq C_{\alpha_{\text{ref}}}$ whenever $\alpha \le \beta \le \alpha_{\text{ref}}$. Assume at least k valid survivors, fixed refined scores and one deterministic total order for final ties. Each result is the exact top-k in that order. Then

$$|\mathcal{R}_\alpha \cap \mathcal{R}_{\text{ref}}| = |C_\alpha \cap \mathcal{R}_{\text{ref}}|, \qquad D_S(\alpha) \ge D_S(\beta). \tag{6.10}$$

The latter statement holds when these conditions hold for every sampled query.

Proof.
For $x \in \mathcal{R}_{\text{ref}} \cap C_\alpha$, the common total order gives

$$|\{y \in C_\alpha : y \prec x\}| \le |\{y \in C_{\alpha_{\text{ref}}} : y \prec x\}| < k.$$

Hence $x \in \mathcal{R}_\alpha$. Together with $\mathcal{R}_\alpha \subseteq C_\alpha$, this proves the intersection identity in equation (6.10). Nestedness gives $|C_\alpha \cap \mathcal{R}_{\text{ref}}| \le |C_\beta \cap \mathcal{R}_{\text{ref}}|$. Summing missing reference counts over queries proves $D_S(\alpha) \ge D_S(\beta)$. □

Fixed-key exact primary top-c gives nesting; fixed-key refinement then gives monotone agreement. This supports bisection, without making GT recall or runtime monotone as the pool grows.

Case	C_1	C_2	C_3	Reference overlap
Nested	$\{b\}$	$\{b, c\}$	$\{a, b, c\}$	0, 0, 1
Non-nested	$\{a\}$	$\{b, c\}$	$\{a, b, c\}$	1, 0, 1

Figure 6.3. Constructed $k = 1$ example with refined order $a \prec b \prec c$ and reference $\{a\}$. The nested case preserves reference membership. With all primary scores tied, different tie choices can produce the non-nested middle pool and break monotonicity. These are explanatory sets, not measured GPU outputs.

6.3.3 A corollary for the evaluated grouped path

Within a draw, S, routing and scan threads are fixed; `scheduled_groups(S)` ignores c. Grouped budgets use the same LUT sums. Fixed B_r, 16 lanes and final $(D_{\mathrm{H}}, \mathrm{ID})$ order keep candidate scores fixed.

Corollary 6.2 (Grouped calibration)**.**
For the evaluated grouped configuration, under the representation and shared-bound contracts of Chapters 4 and 5, primary survivors are nested across budgets. Consequently Proposition 6.1 applies and bisection returns the same minimum accepted grid budget as enumeration.

Proof.
Lemma 5.1 and propositions 5.1 and 5.2 give exact primary top-c under the common key $(D_{\mathrm{P}}, \mathrm{position})$. Its prefixes nest as c grows. Fixed refinement/final order supplies proposition 6.1's remaining premises. □

The corollary covers 2,176 draws; 128 OpenAI-3072/S = 128 draws use distance-only selection. All 2,304 recorded curves are monotone and match enumeration (section 7.5.5); this is finite evidence.

Full-pool coverage. Two FULL cells group Vogue; ten use $g_{\mathrm{scan}} = 0$. All twelve match enumeration. Ten distance-only cells remain outside corollary 6.2: nesting needs a common cutoff order or no crossing ties. A matching held-out path does not ensure this.

A rank-based extension. Under the same nesting/fixed-score premises, one-based primary ranks give

$$D_S(\alpha) = \sum_{q \in \mathcal{Q}_S} \sum_{x \in \mathcal{R}_{\mathrm{ref}}(q)} \mathbf{1}\{\mathrm{rank}_{\mathrm{P}}(x) > c(\alpha)\}. \tag{6.11}$$

Exporting reference ranks could replace repeated budget searches. This is unimplemented.

6.4 Budget Selection Baselines

Table 6.1 separates budget selection from its evaluation. Fixed arms share the index, route and held-out queries; invocation and timing details are in section 3.5.2.

Table 6.1. Budget controls and the questions they answer. Held-out GT is used to assess every result, but only the diagnostic oracle uses it to choose a budget. The reference is fixed at 200 for agreement-based arms.

Control	Selection information	Question
Fixed 100	No calibration	Gain relative to the inherited default?
Fixed 64 and smaller grid arms	No calibration	Does a simple fixed choice remove the need for calibration?
Sampled bisection	S unlabelled queries; slot agreement	How well does a low-cost sample choose a reusable budget?
Same-sample enumeration	Same S, reference and threshold; all grid arms	Did bisection miss the smallest accepted grid budget?
Full-pool calibration	All 500 calibration queries; same bisection rule	What changes with broader coverage and a stricter normalised allowance?
Held-out oracle	GT recall and timed QPS for all arms	Which measured arm is fastest under the diagnostic quality constraint?

For held-out recall/QPS $\widehat{r}, \widehat{Q}$ and $\widehat{r}^* = \max_{\alpha \in A} \widehat{r}(\alpha)$,

$$\alpha_{\text{oracle}} \in \arg\max_{\alpha \in A} \left\{ \widehat{Q}(\alpha) : \widehat{r}(\alpha) \geq \widehat{r}^* - \delta \right\}, \qquad \delta = 10^{-3}. \tag{6.12}$$

This labelled/timed diagnostic uses recall slack δ, not slot allowance ϵ.

FULL tightens $\epsilon/(Sk)$ without labels; enumeration checks search correctness. Agreement minimises accepted budget, while the oracle maximises quality-constrained speed.

Fixed 64/200 test smaller/high budgets (section 7.5 and table 7.9).

6.5 Calibration Quality and Reuse Cost

6.5.1 Choosing a calibration scale

The strong independent-slot model suggests

$$\frac{4.74}{Sk} \simeq 10^{-3}, \quad k = 10 \quad \Longrightarrow \quad S \simeq 474 \approx 500.$$

This resolves reference disagreement, not GT risk (section 6.2). FULL quality and sampled risks are in table 7.8 and figure 7.6.

FULL still needs the path/order conditions in section 6.3.

6.5.2 Risk and selected-arm performance

Over m calibration draws, let draw r select $\widehat{\alpha}_r$. Define held-out loss and threshold-crossing frequency by

$$L_r = \widehat{r}^* - \widehat{r}(\widehat{\alpha}_r), \qquad \widehat{p}_\delta = \frac{1}{m} \sum_{r=1}^{m} \mathbf{1}\{L_r > \delta\}. \tag{6.13}$$

Mean loss, crossing frequency and tail size answer different questions. Table 7.9 and figure 7.6 report the archived summaries; the largest supplementary per-draw losses are reported in section 7.5.5.

Reported selected-arm rate is $\overline{Q} = m^{-1} \sum_r \widehat{Q}(\widehat{\alpha}_r)$, a lookup from timed fixed arms. It is distinct from the median budget's rate and from the throughput of a calibrated query stream. Quality and speed must be read together (table 7.10).

6.5.3 Cost of reuse

For B_{reuse} equal-sized subsequent batches, model

$$T_{\text{policy}}(B_{\text{reuse}}) = T_{\text{cal}} + B_{\text{reuse}} T_{\text{chosen}}, \qquad T_{\text{fixed}}(B_{\text{reuse}}) = B_{\text{reuse}} T_f. \tag{6.14}$$

When $T_{\text{chosen}} < T_f$, the break-even count is

$$B^*_{\text{reuse}} = \frac{T_{\text{cal}}}{T_f - T_{\text{chosen}}}. \tag{6.15}$$

Indeed,

$$\begin{aligned} T_{\text{policy}} < T_{\text{fixed}} &\iff T_{\text{cal}} < B_{\text{reuse}}(T_f - T_{\text{chosen}}) \\ &\iff B_{\text{reuse}} > B^*_{\text{reuse}} \qquad (T_{\text{chosen}} < T_f). \end{aligned}$$

If $T_{\text{chosen}} \geq T_f$, there is no throughput payback. Batch time is $T = 1000n/Q$ milliseconds. For varying choices, convexity gives $1/\mathbb{E}[Q] \leq \mathbb{E}[1/Q]$: inverse mean QPS understates mean time. Boundaries/direct checks follow sections 3.5.2 and 7.5.6. Campaign estimates are in table 7.10.

Periodic calibration adds T_{cal}/H per batch. Automatic drift detection/renewal is future work.

6.6 Calibration Reuse and Applicability

Reuse needs stable index/k/routing/queries (section 6.3); quality/tails/cost follow section 6.5. Drift detection is unimplemented.

The caller must explicitly install α before reuse.

Budget evidence uses v69/OpenAI TF32; FP32 frontier checks do not recalibrate it.

7 Experimental Evaluation

We test construction/capacity (RQ1), execution (RQ2), budget quality/reuse (RQ3) and competitive regions (RQ4).

7.1 Experimental Protocol

7.1.1 Comparison protocol and reproducibility

Sections 3.2, 3.3 and 3.5 define the protocol. Search excludes construction/calibration; each ablation has its own baseline.

CPU JHQ uses unmatched settings/protocols: October Vogue has 1,024 lists and $[8, 4]$ bits versus E1's 4,096 lists. A matched CPU/GPU speedup is unavailable.

Adaptive calls/device-only timing are in section 3.5.2.

Metrics and frontier estimators follow sections 3.2 and 3.4; g_{LUT} and g_{scan} retain the distinct meanings in section 5.1.

7.2 Cross-System Recall–Throughput Comparison

7.2.1 Comparison scope and frontier construction

Figure 7.1 compares JHQ-GPU with the graph, compressed-IVF and brute-force baselines of section 3.4 [17, 28, 33], using the first four datasets in table 3.2. E1 uses batch 1,000/common GT; T5 covers small batches (section 7.2.6), with internal ablations in section 7.6.

E1 dispatches 105 arXiv/OpenAI cells to $g_{\mathrm{scan}} = 0$ and 27 Vogue cells to $g_{\mathrm{scan}} = 4$, triggered by a coefficient of variation (CV) of 0.338–0.622. Only Vogue exercises grouped retention/bounds/merging (section 5.5).

E1 has 1,590 valid points: JHQ 27–39/dataset, baseline families 54–264. No failure/invalid-ID/duplicate-row flags occur. Sweeps share indexes (section 3.4).

Table 3.3 and section 3.4 define finite coverage; targets beyond it remain unmeasured.

Storage differs: nine-bit RaBitQ shares $9d/8$ nominal code bytes; 4/8-bit codes use less. Table 4.1 and section 7.7 give representation costs and measured allocations alongside this performance envelope.

7.2.2 Throughput at matched recall

Table 7.1 reports recall 0.97 using the interpolated and qualifying-measured estimators defined in section 3.4.

Table 7.1. Recall 0.97 comparison. Each ratio is interpolated / measured; values above one favour JHQ. JHQ kQPS is the interpolated estimate. The small arXiv–RaBitQ difference changes direction with the estimator.

Dataset	JHQ kQPS	vs CAGRA fp32	vs IVF-RaBitQ
Vogue	230.8	1.416 / 1.537	1.059 / 1.046
arXiv	258.6	1.317 / 1.422	0.996 / 1.015
OpenAI-1536	126.7	1.747 / 2.294	1.130 / 1.111
OpenAI-3072	55.9	1.747 / 2.088	1.157 / 1.346

Removing grouping lowers Vogue's JHQ/RaBitQ ratio at recall 0.97 from 1.059 to 0.933 at fixed index/probes/budget/final outputs (batch 1,000). This isolates the integrated grouping path, not bounds alone.

CAGRA int8 leads 1.20–1.98× at 0.90–0.92, without 0.97 coverage. RaBitQ wins some 0.98+ targets; CAGRA fp32 wins near 0.99 on arXiv (figure 7.1).

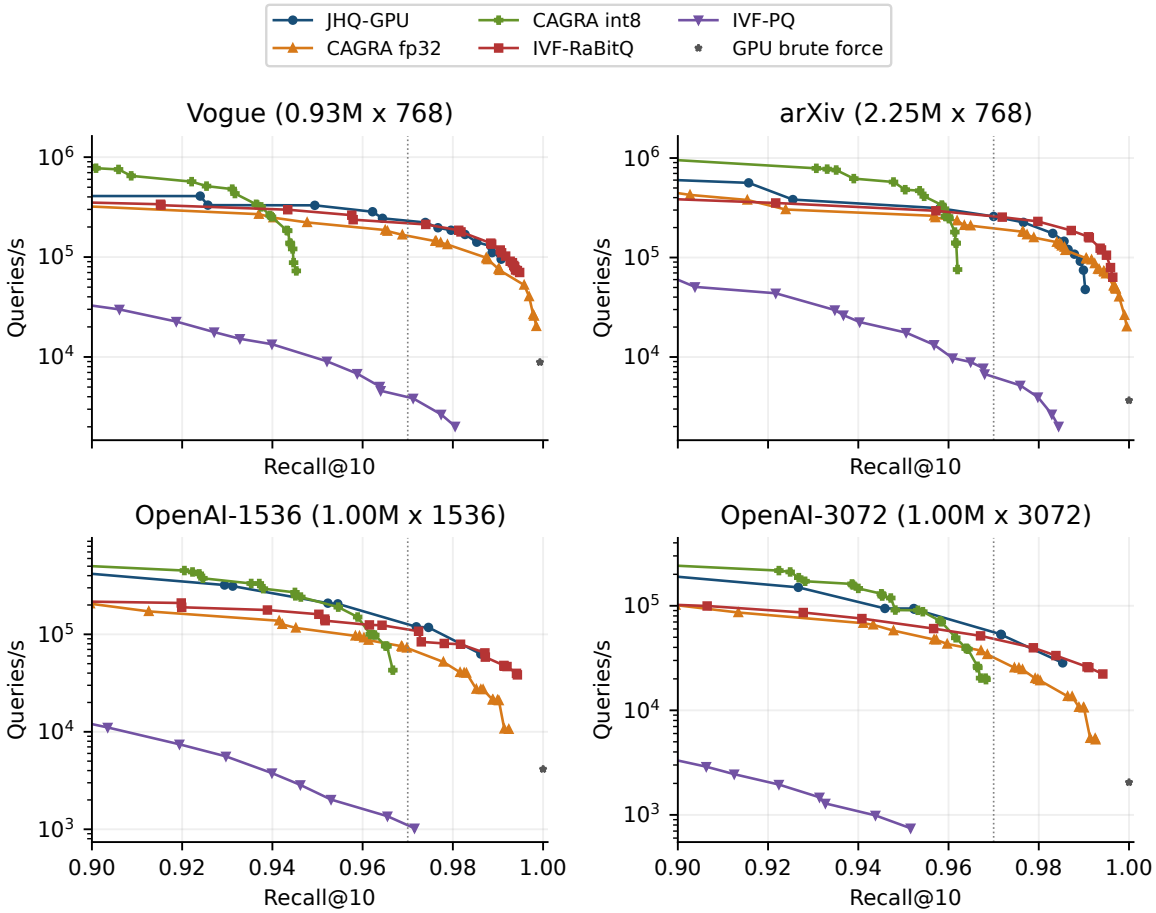


Figure 7.1. Measured host-to-host recall–QPS frontiers on the four E1 datasets. The displayed recall window begins at 0.90, with a vertical line at 0.97; all baseline families, including CAGRA int8, participate in frontier construction. Lines connect measured points; throughput uses a logarithmic axis.

7.2.3 Precision sensitivity on the OpenAI datasets

Supplementary source inspection establishes that the E1 implementation uses TF32 multiplication with FP32 accumulation for OpenAI rotation and centroid GEMMs, despite FP32 input/storage and a misleading source comment (section 4.2). The main E1 frontier remains the historical measurement of that implementation. In campaign D, a separate FP32 variant changes the cuBLAS math-mode switch and replays 37/39 OpenAI configurations at batch 1,000, five rounds each. Execution order alternates between variants across probe settings. Both reuse the same TF32-trained model; the intervention changes add/query arithmetic, including database encoding and routing. It is not a comparison of independently FP32-trained models. Table 7.2 summarises the paired changes.

Table 7.2. FP32 sensitivity. QPS ratio is FP32/TF32 at equal configuration (geometric mean); recall differences have the same direction. The last two columns compare the FP32 frontier at recall 0.97 with archived E1 IVF-RaBitQ and CAGRA fp32 frontiers. Campaign D does not remeasure the baselines.

Dataset	Pairs	QPS ratio	ΔRecall	vs RaBitQ	vs CAGRA
OpenAI-1536	37	0.953	+0.0003–+0.0005	1.086	1.679
OpenAI-3072	39	0.951	+0.0000–+0.0003	1.079	1.629

At equal configuration, FP32 reduces throughput by about 5% and increases recall by 0–0.0005. At recall 0.97, FP32 is about 3% slower than same-session TF32, yet 1.086/1.079× archived RaBitQ and 1.679/1.629× archived CAGRA fp32. New-session v69 gives 1.118/1.114 against archived RaBitQ, versus historical 1.130/1.157. D compares baselines across sessions;

only v69/v72 share a session. Figure 7.2 shows the two JHQ frontiers with their archived comparators. T1 separately remeasures same-session anchors, with grid-sensitive results in section 7.2.4.

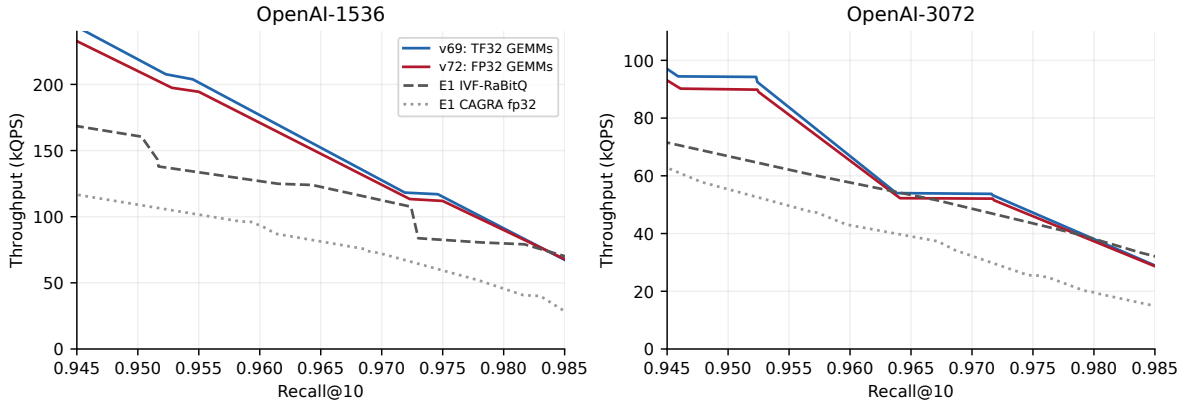


Figure 7.2. Campaign D precision frontiers. Solid curves are same-session JHQ variants; dashed/dotted baselines are historical E1 measurements. All use the chapter's frontier and interpolation rules.

7.2.4 Same-session baseline comparisons

T1 uses E1 pools and twelve paired configurations per OpenAI dataset. Two five-round JHQ repetitions bracket baseline runs; medians combine geometrically. Equal-configuration FP32/TF32 means are 0.956/0.964. Both reuse TF32-trained caches, changing add/query arithmetic.

Table 7.3. T1 at recall 0.97, batch 1,000. Baseline ratios are interpolation / strict measured threshold; FP32/TF32 is interpolated. All methods share a session, unlike D's archived baselines.

Dataset	FP32/TF32 at 0.97	FP32 / RaBitQ	FP32 / CAGRA
OpenAI-1536	0.980	1.126 / 1.053	1.752 / 1.633
OpenAI-3072	0.973	1.328 / 1.316	1.825 / 2.019

The OpenAI-3072 RaBitQ bracket (0.9669–0.9796 recall) shows grid sensitivity. These anchors neither replace E1 nor provide FP32 retraining.

7.2.5 Interpretation and comparison limits

Both OpenAI datasets use Qdrant DBpedia embeddings [31, 32]. After truncation/renormalisation, 1,000 query and 10,299 sampled base rows have cosine $\geq$ 0.9980 (median 1.000000), with GT overlap 9.046/10. Without entity IDs, these checks support row alignment. Doubling width leaves CAGRA ratios at 1.747 and RaBitQ at 1.130/1.157, so width alone is insufficient.

Gains accompany smaller 128-probe anchors: OpenAI 4–8, Vogue 64, arXiv 100 (later FULL 200). This is an association, not an isolated effect or matched-frontier setting.

Section 7.8 checks sensitivity to small margins and the interface/memory/compilation differences of section 3.3.

7.2.6 Small batches change the competitive region

T5 compares scheduled JHQ, CAGRA fp32/int8 and IVF-RaBitQ at batches 8/32 on the four E1 datasets. Five synchronous host-to-host rounds use 1,000/992 queries, respectively. The 1,808 points include a JHQ one-block control, excluding IVF-PQ/brute force. Figure 7.3 and table 7.4 compare the resulting regions.

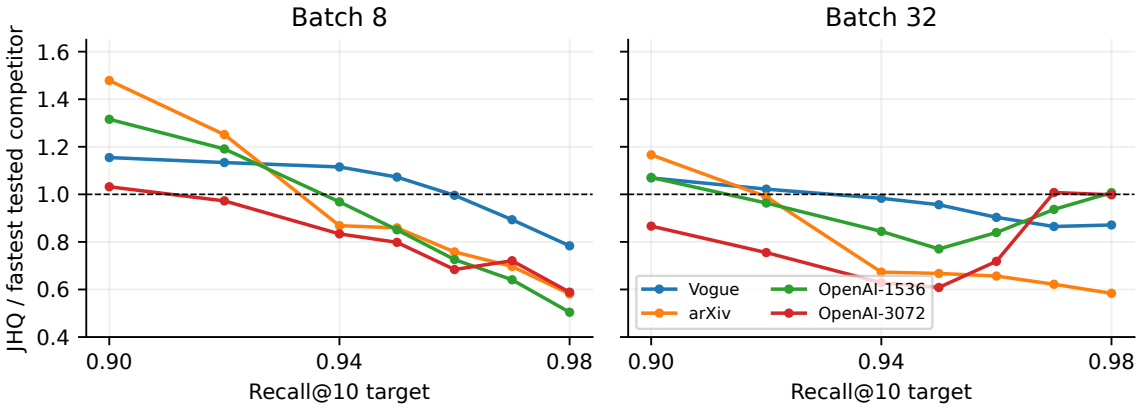


Figure 7.3. T5 JHQ / fastest-competitor QPS under interpolation with qualifying measured endpoints. Above one favours JHQ. Table 7.4 gives strict thresholds.

Table 7.4. T5 QPS ratios: interpolation with qualifying endpoints / strict measured threshold. The fastest competitor is selected independently for each estimator; higher-recall qualifying endpoints remain eligible without extrapolation.

Dataset	b	$R = 0.90$	$R = 0.95$	$R = 0.97$
Vogue	8	1.155 / 1.091	1.073 / 0.980	0.893 / 0.854
Vogue	32	1.069 / 0.997	0.956 / 0.895	0.865 / 0.839
arXiv	8	1.479 / 1.454	0.859 / 0.851	0.697 / 0.698
arXiv	32	1.166 / 1.148	0.667 / 0.661	0.622 / 0.582
OpenAI-1536	8	1.316 / 1.131	0.851 / 0.825	0.641 / 0.620
OpenAI-1536	32	1.071 / 0.915	0.770 / 0.752	0.937 / 0.914
OpenAI-3072	8	1.032 / 0.953	0.798 / 0.798	0.721 / 0.702
OpenAI-3072	32	0.866 / 0.734	0.608 / 0.771	1.008 / 1.351

At batch 8/recall 0.97, CAGRA fp32 wins all four datasets under both estimators (JHQ ratios 0.641–0.893). At batch 32, ratios are 0.622–1.008; OpenAI-3072 favours JHQ under the measured threshold. Vogue's batch-8/0.95 lead requires interpolation. Grid resolution changes near ties.

Including qualifying endpoints, recall-0.90 ratios are 1.032–1.479 (batch 8), 0.866–1.166 (32), with three/one strict JHQ wins. Omitting endpoints overstates gains. Scheduling gains 1.7–3.3× over one block at 0.95, without ensuring cross-system wins.

Null-call medians are 0.012–0.072 ms and remain included. CAGRA uses Python, JHQ/RaBitQ C++; warm-ups differ (section 3.5.2).

7.3 Cartesian LUT, Packed Loads and Lane Refinement

The RQ2 ablations measure both kernel and application effects.

7.3.1 LUT factorisation and throughput

The LUT ablation tests the cost model in section 5.2 and FP32 ranking effects.

Figure 7.4 compares Cartesian table grouping on six datasets, probe counts 8/32/128/512, fixed $\alpha = 100$, and $g_{\mathrm{LUT}} \in \{1, 2, 4, 8\}$. Five rounds use batch 1,000 (Stella 999). Two groups lead in 22/24 cells. Exceptions are near ties: arXiv/128 favours one group by 0.23%; OpenAI-3072/8 favours four by 0.84%. Paired two-versus-one wins are 23/24; all dataset geometric means exceed one (1.021–1.318). Shared-index cells are not independent trials.

The two-group/one-group ratio ranges from 0.998 to 1.914. Four and eight groups each store 16 entries but require more lookups and additions. These arms show that smaller tables need not improve throughput; occupancy and instruction costs were not isolated. The maximum reported recall spread within any four-arm cell is 10^{-4}. At 128 probes and $\alpha = 100$, T4



Figure 7.4. Cartesian table grouping at fixed $\alpha = 100$ (v71). Group counts describe g_{LUT}, not parallel scan groups. Curves are medians of per-dataset QPS ratios to the one-group arm; shading shows their range.

compares identical candidates on Vogue, arXiv and OpenAI-3072. Full versus $16 + 16$ tables differ in 22.6–26.1% of candidate-score bit patterns, with maximum relative error 4.53×10^{-7}. Across all forms, 20.5–29.4% differ, with maximum relative error 4.64×10^{-7}. Vogue has 2–4/256 final-ID differences, while arXiv/OpenAI have none. Unchanged primary survivor sets can still produce different final IDs because the rounded primary score enters refinement (section 3.5.2).

7.3.2 Packed access and cooperative residual scoring

Packing wins 1.012–1.658× across 48 matched pairs (six datasets, four probe counts, $g_{LUT} \in \{1, 2\}$), with recall differences at most 10^{-4}. Both layouts are subspace-major. Packing/load/unpack change together. One aggregate per arm gives less timing evidence than five-round v71.

Figure 7.5 and table 7.5 then separate wider loads from lane cooperation in residual refinement. L0 is the original scalar path; L1 changes the load path without distributing a candidate across multiple lanes; L16 assigns 16 lanes to a candidate. The same process rotates arm order across five timing rounds, with three search calls per arm per round. Across twelve ordinary cells, L16 gains 1.015–1.789× overall. The separate high-budget arXiv/$\alpha = 200$ control reaches 2.383×.

Table 7.5. Refinement at 128 probes. Ratios are paired end-to-end gains over L0, except the last column, which is the profiled refinement-kernel gain. The kernel and application gains must not be interchanged.

Dataset	α	$L1/L0$ QPS	$L16/L0$ QPS	Kernel gain
arXiv	100	1.368	1.789	5.97
BGE	32	1.026	1.062	1.97
OpenAI-1536	8	1.055	1.145	3.34
OpenAI-3072	4	1.063	1.171	8.81
Stella	16	1.030	1.053	2.66
Vogue	64	1.161	1.240	3.72

Load-only L1 matches L0 bits yet gains 1.368× on arXiv. With lanes at $\alpha = 100$, refinement drops 4,277.5→716.8 μs (5.97×) versus 1.789× overall. OpenAI-3072 gains 8.81× in-kernel/1.171× overall. Other stages remain.

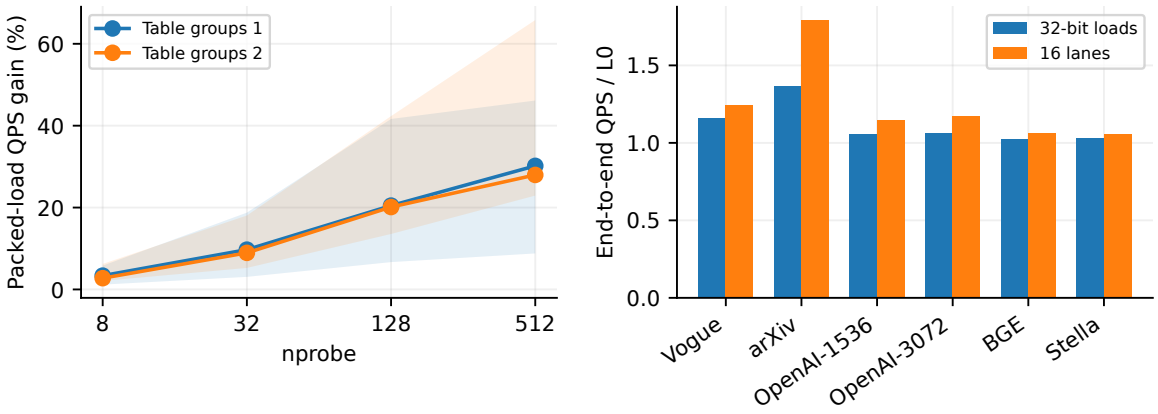


Figure 7.5. Packing and refinement controls. Left: median packed-load QPS gain across six datasets, with shaded minimum–maximum ranges, for each table grouping and probe count. Right: measured load-only and 16-lane end-to-end gains at 128 probes and the ordinary dataset-specific budgets. Panels use different experiment families.

For baseline wall-time fraction f and refinement speedup s, the replacement model gives

$$\frac{T'}{T} = (1 - f) + \frac{f}{s}, \qquad \frac{T}{T'} = \frac{1}{1 - f + f/s}.$$

ArXiv/OpenAI-3072 $f = 0.529/0.168$ predicts $1.787/1.175\times$, versus measured $1.789/1.171\times$. Separate profiling gives an explanatory estimate.

Lane FP32 reassociation reaches relative distance error $\approx 1.6 \times 10^{-6}$ and occasionally changes order. Sanitizer coverage is limited to recorded subsets; general bit identity is not claimed.

7.4 Selection Correctness and Performance

7.4.1 Strict CPU reference for computed-score selection

T4 sorts GPU-produced scores by $(D_{\mathrm{P}}, \text{position})$ on CPU. The exporter follows scan layout/accumulation and matches valid returned scores bitwise; source comparison finds only export changes from the table-sweep version. Its 270 configurations cover Vogue/arXiv/OpenAI-3072, probes 1/8/128, $\alpha \in \{4, 100, 200\}$, $g_{\mathrm{scan}} \in \{0, 1, 2, 4, 8\}$ and both bound settings: 69,120 instances (table 7.6).

Table 7.6. T4 CPU-reference checks. Set and order equality use the complete primary key. Valid count checks $\min(c, n_c)$ survivors; final equality sorts retained computed refined scores by $(D_{\mathrm{H}}, \mathrm{ID})$, including their FP32 bits. Counts are query instances, with queries reused across configurations.

Path	Queries	Set / order exact	Tie-only	Valid count	Final exact
$g_{\mathrm{scan}} \geq 1$	55,296	55,296 / 55,296	0	55,296	55,296
$g_{\mathrm{scan}} = 0$	13,824	13,768 / 8,758	56	13,824	13,824

Table 7.6 confirms grouped full-key order and one-block distance order (56 tie-only set differences). All 19,120 underfull counts and 69,120 retained-score final selections pass, testing lemma 5.1 and propositions 5.1 and 5.2. Scan/bound and sanitizer coverage remains separate (section 3.5.2).

7.4.2 Shared bounds preserve the selection target

All 48 outputs agree; QPS ratios span 0.994–1.562×. At arXiv $\alpha = 200$, batch 1,000 and eight groups, sharing cuts merge 2,629→471 μs and scan 7,968→5,513 μs. Some cells regress slightly.

The pilot's first group holds 46–81% of winners. Equal quotas need fallback; unequal ones retain a bottleneck. Bounds avoid fixed quotas.

7.4.3 Probe selection and output consistency

Radix retains score arithmetic/OpenAI TF32 (section 7.2.3). All 1,792 synthetic cases pass. Probes and primary scores/positions match on all 1,000 arXiv/Vogue/OpenAI-3072 queries.

Table 7.7. v69/v65 end-to-end throughput ratios for the exact computed-score probe change. Each workload–batch comparison has ten timing rounds per binary, collected in alternating runs. Ratios > 1 favour v69.

Dataset	Batch 8	32	256	1,000
arXiv	1.074	1.063	1.035	1.034
Vogue	1.109	1.103	1.052	1.041
OpenAI-3072	1.167	1.117	1.055	1.050

Table 7.7 shows end-to-end improvements of 1.034–1.167×. At batch 8, the profiled probe stage falls from 56.5 to 22.6 μs on arXiv, 52.7 to 14.7 μs on Vogue, and 77.8 to 16.7 μs on OpenAI-3072. Unchanged stages limit full-batch gains; each ratio uses its own baseline.

Vogue probe/primary dumps match bytewise on 1,000 queries across binaries (one-group/batch-8), yet final recall differs 0.9642/0.9644. The baseline alone also changes from 0.9642 to 0.9644 across identity/timing runs. Variation follows primary selection in this check and is unresolved. Within-process scheduled/one-block outputs match (section 3.5.1).

Optional FP16 probe sets match v69 on 936/979/998 of 1,000 arXiv/Vogue/OpenAI-3072 queries. High overlap does not establish exactness. Those results are excluded from the main E1 frontier and from the exact-probe claim.

7.5 Held-out Budget Quality, Throughput and Payback

RQ3 compares the policy of chapter 6 with fixed $\alpha = 100$ and the stronger universal control $\alpha = 64$.

7.5.1 Unlabelled selection and a labelled diagnostic oracle

The six datasets are tested at 128 and 512 probes with the fixed, sampled, FULL, enumeration and oracle controls in section 6.4. The disjoint pools and 64 draws per sample size follow section 3.2.

All rule variants use equation (6.4) with $k = 10$ and $\epsilon = 1$. FULL uses all calibration queries; the diagnostic oracle uses held-out labels (table 6.1).

7.5.2 Full-pool calibration as the quality-first working point

Full-pool calibration uses all 500 calibration queries once per cell. Its selected arms meet the held-out 10^{-3} loss target in 12/12 cells; the largest observed loss is 0.0006. Their rates are 0.908–1.000 of the GT oracle's, and calibration costs 9.5–60.3 ms. The geometric-mean rate ratio to fixed 100 is 1.111×, or 1.193× over the ten cells where fixed 100 also meets the target. Table 7.8 shows the complete comparison. Against fixed 64, full-pool selection is faster in six cells, equal in two and slower in four: its main advantage is the observed coverage of all twelve quality targets.

These cells favour FULL for quality and $S = 128$ for calibration cost (section 6.2).

7.5.3 Lower-cost sampling and its quality risk

Table 7.9 reports the $S = 128$ result. The median selected budget matches the oracle budget in ten of twelve cells; it differs on arXiv at 128 probes and OpenAI-3072 at 512. Budgets span 4–200 across workloads. Median budget and mean recall need not describe one arm. Mean recall uses printed best recall minus mean loss, approximately to four decimals.

Table 7.8. Full-pool calibration ($S = 500$). Loss is relative to the best measured held-out recall; rates are kQPS from the selected fixed arm. "Cal." is one full-pool calibration call per cell, not a repeated stream timing. Held-out counts are 500, or 499 for Stella.

Dataset	Probe	α_{full}	Loss	Q_{full}	vs 100	vs oracle	Cal. ms
Vogue	128	100	0.0000	202.9	1.000	0.908	10.1
arXiv	128	200	0.0000	151.5	0.728	1.000	10.0
OpenAI-1536	128	8	0.0000	267.7	1.599	1.000	9.5
OpenAI-3072	128	4	0.0000	145.0	1.691	1.000	17.1
BGE	128	64	0.0006	124.9	1.075	0.932	17.1
Stella	128	16	0.0006	110.4	1.153	1.000	20.6
Vogue	512	100	0.0002	100.1	1.000	0.969	20.9
arXiv	512	200	0.0000	69.0	0.831	1.000	25.1
OpenAI-1536	512	8	0.0000	114.9	1.326	1.000	21.8
OpenAI-3072	512	4	0.0002	52.1	1.219	0.977	40.5
BGE	512	64	0.0006	50.8	1.033	1.000	42.3
Stella	512	16	0.0006	36.0	1.041	1.000	60.3

Table 7.9. Held-out quality at $S = 128$. Rule α is the median across 64 draws; oracle α uses held-out labels. $\overline{R}_{\text{rule}}$ is reconstructed mean recall. "Losses" counts draws with recall loss $> 10^{-3}$ relative to the best tested held-out recall; L_{95} is the logged lower-order 95th-percentile loss estimate. R_{100} is fixed-budget recall.

Dataset	Probe	α: rule/oracle	R_{100}	$\overline{R}_{\text{rule}}$	Losses	L_{95}
Vogue	128	64 / 64	0.9680	0.9677	5/64	0.0022
arXiv	128	100 / 200	0.9748	0.9751	40/64	0.0024
OpenAI-1536	128	8 / 8	0.9280	0.9279	4/64	0.0000
OpenAI-3072	128	4 / 4	0.9252	0.9252	0/64	0.0000
BGE	128	32 / 32	0.9198	0.9190	1/64	0.0008
Stella	128	16 / 16	0.9892	0.9884	6/64	0.0030
Vogue	512	64 / 64	0.9916	0.9915	2/64	0.0002
arXiv	512	200 / 200	0.9878	0.9885	32/64	0.0016
OpenAI-1536	512	8 / 8	0.9740	0.9738	4/64	0.0000
OpenAI-3072	512	4 / 8	0.9704	0.9702	0/64	0.0002
BGE	512	64 / 64	0.9670	0.9661	24/64	0.0014
Stella	512	16 / 16	0.9954	0.9945	7/64	0.0030

ArXiv has 40/64 and 32/64 loss crossings at 128/512 probes, despite mean loss 0.0009. Fixed 100 loses 0.0012/0.0016: better than default need not meet the target. BGE/512 has 24 crossings and $L_{95} = 0.0014$; Stella reaches 0.0030; OpenAI-3072 has none (table 7.9). Four losses can coexist with reported $L_{95} = 0$ (section 3.5.2).

7.5.4 Estimated throughput and calibration economics

With section 3.5.2's mean selected-arm estimate, $S = 128$ beats fixed 100 in ten cells (1.028–1.691×). ArXiv gives 0.908/0.917 with better mean recall: no throughput payback.

Against fixed 64, sampling wins 9/12 cells: geometric mean 1.054×, range 0.821–1.367× (OpenAI 1.052–1.367×). Both fixed controls fail arXiv quality, preventing a uniformly quality-matched comparison.

Calibration costs 3.9–21.3 ms. Maximum positive estimated paybacks against fixed 100/64 are 0.54/2.72 s, or FULL 1.52/2.25 s (section 6.5, equation (6.15), and table 7.10). ArXiv controls fail quality; direct reuse below measures both cost and recall.

7.5.5 Complete decision traces and finite-pool risk

All 2,304 sampled and twelve FULL curves are monotone with enumeration-matched choices. Recorded subset sums reproduce 18,432 grid counts (equation (6.9)). This verifies recorded

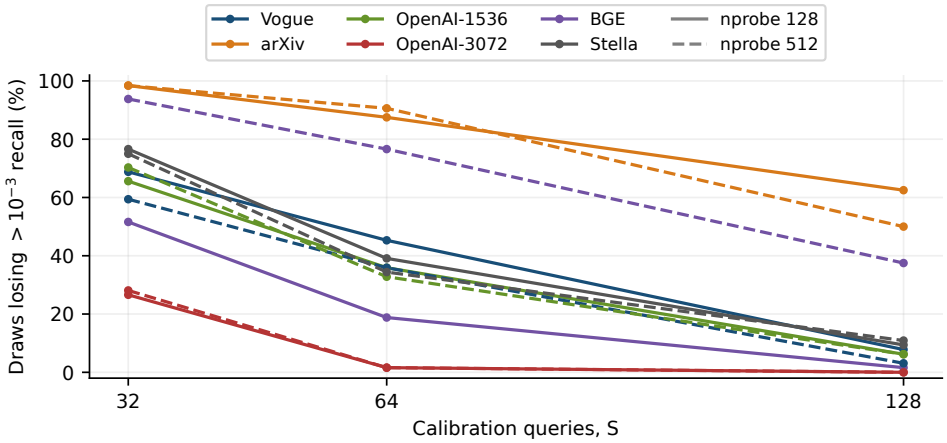


Figure 7.6. Held-out risk across sample sizes. The vertical axis is the percentage of calibration draws losing more than 10^{-3} held-out recall relative to the best tested budget. Increasing S also tightens the allowed slot-disagreement fraction because equation (6.4) keeps threshold one.

Table 7.10. Calibration economics, $S = 128$. Rates are kQPS and rule rate is mean selected-arm QPS. Ratios compare with fixed 100 and fixed 64. The "Batches" column gives payback relative to fixed 100 using 500 queries, or 499 for Stella; "Never" denotes no throughput payback. The quality column separately marks whether fixed 100 meets the held-out loss target.

Dataset	Probe	Q_{100}	$\overline{Q}_{\text{rule}}$	vs 100	vs 64	Cal. ms	Batches	Fixed 100 quality
Vogue	128	202.9	224.4	1.106	1.004	4.0	17.0	Pass
arXiv	128	207.9	188.8	0.908	0.821	4.2	Never	Fail
OpenAI-1536	128	167.5	268.5	1.603	1.367	3.9	3.5	Pass
OpenAI-3072	128	85.7	145.0	1.691	1.230	5.7	2.4	Pass
BGE	128	116.2	131.2	1.130	1.051	6.5	13.2	Pass
Stella	128	95.7	110.3	1.153	1.087	7.5	10.8	Pass
Vogue	512	100.1	102.9	1.028	0.996	7.5	55.4	Pass
arXiv	512	83.1	76.2	0.917	0.870	9.7	Never	Fail
OpenAI-1536	512	86.6	115.0	1.327	1.268	8.8	6.2	Pass
OpenAI-3072	512	42.8	52.1	1.219	1.052	13.1	6.2	Pass
BGE	512	49.2	51.1	1.039	1.006	15.8	41.2	Pass
Stella	512	34.6	36.0	1.041	1.028	21.3	37.1	Pass

aggregates; arbitrary subset/position invariance and general distance-only nesting remain conditional (section 6.3).

Maximum loss is 0.0050 (arXiv/512), 0.0046 (BGE/512) at $S = 128$, and 0.0294 at $S = 32$. FULL stays within 0.0006.

ArXiv/128 predicted/observed crossings are 0.965/0.984, 0.875/0.875, 0.620/0.625 at $S =$ 32/64/128 (figure 7.7). The largest of 36 gaps is BGE/512: 0.144 (0.519/0.375). GT-loss prediction needs stable counts/held-out arm quality; calibration uses no labels.

Ten alternating repeats support a low-budget timing plateau, rather than general runtime monotonicity.

7.5.6 Direct calibration-plus-reuse measurement

Four cells compare fixed 64/100, sampling and FULL across five rotated rounds of 500 repeated held-out batches. Timing includes gathering/calibration/budget installation/first graph, excluding build/warm-up/scan calibration (table 7.11).

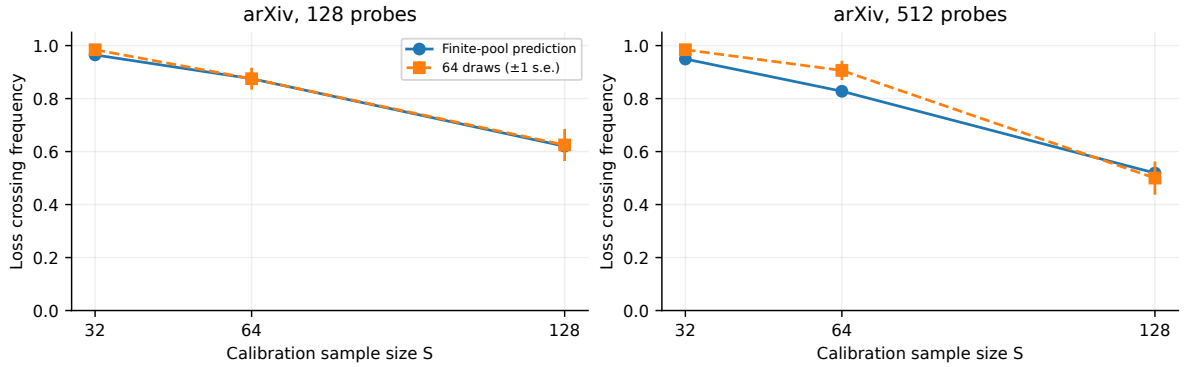


Figure 7.7. Finite-pool prediction and observed arXiv loss crossings. Bars show one estimated standard error over 64 independently drawn subsets conditional on the fixed pool, not uncertainty over future workloads.

Table 7.11. Direct total milliseconds for 500 reuse batches, averaged over five rounds, including budget calibration for sampled and FULL policies. Each batch has 500 queries. Quality is discussed with the timing results.

Dataset/probes	Fixed 100	Fixed 64	Rule $S = 128$	FULL
arXiv/128	1224	1102	1291	1680
OpenAI-1536/128	1491	1264	852	860
OpenAI-3072/128	2757	2160	1769	1783
BGE/512	5185	5018	4921	5062

Both OpenAI-3072 policies select 4 at recall 0.9252, matching fixed 100. OpenAI-1536 FULL/fixed 100 give 0.9280, sampling 0.9276. Table 7.11 gives times.

Direct savings start after 4/6 OpenAI-1536 batches and 4/9 OpenAI-3072 batches versus fixed 100/64 (figure 7.8); fractional fits differ. BGE sampling/fixed 100/FULL recall is 0.9658/0.9670/0.9664. ArXiv FULL selects slower 200 for quality; fixed controls fail.

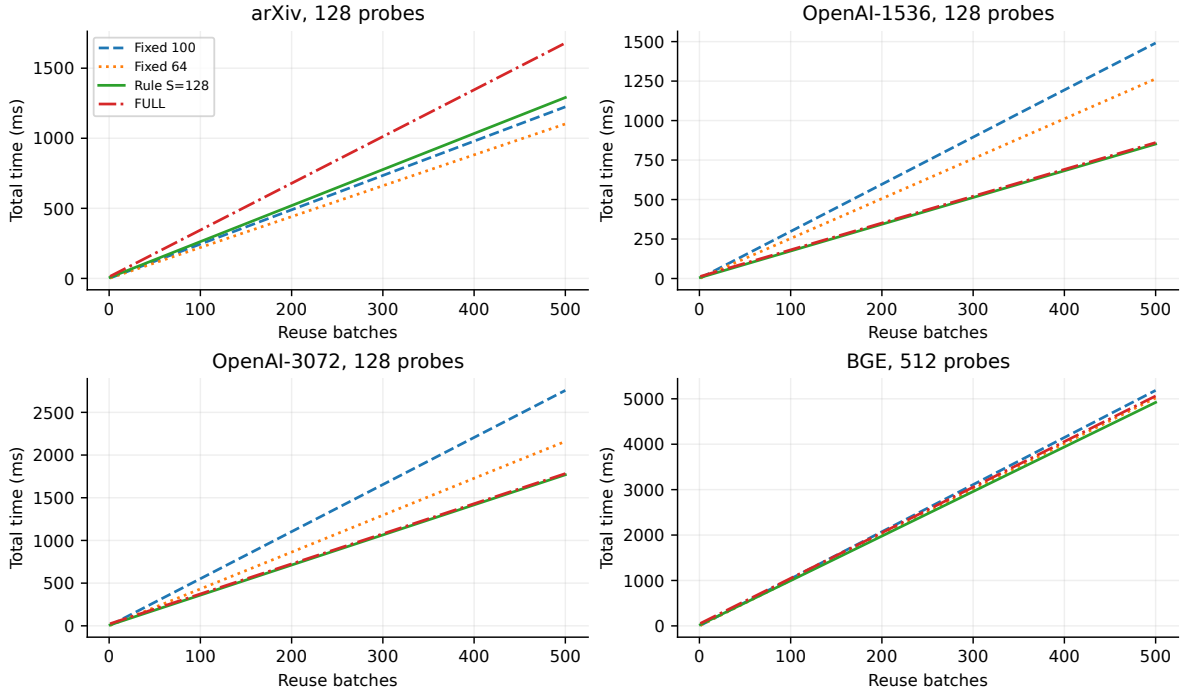


Figure 7.8. Direct mean cumulative time, including budget calibration and first-batch setup. Each curve replays one fixed held-out query pool. Policies can have different recall; timings alone are not a quality-matched ranking. Dataset/probe settings match table 7.11.

7.6 Batch Size, Scheduling and Workload Imbalance

7.6.1 Parallelism versus repeated selection work

Batch-8 eight-group gains are 2.06–2.76×; arXiv scan is 849→216 μs plus 16.7 μs merge. At batch 1,000 and $\alpha = 200$, independent grouping instead raises scan 3,530→7,652 μs plus 2,624 μs merging, reaching only 0.46× one-block QPS. Shared bounds reduce work, without removing this regime change.

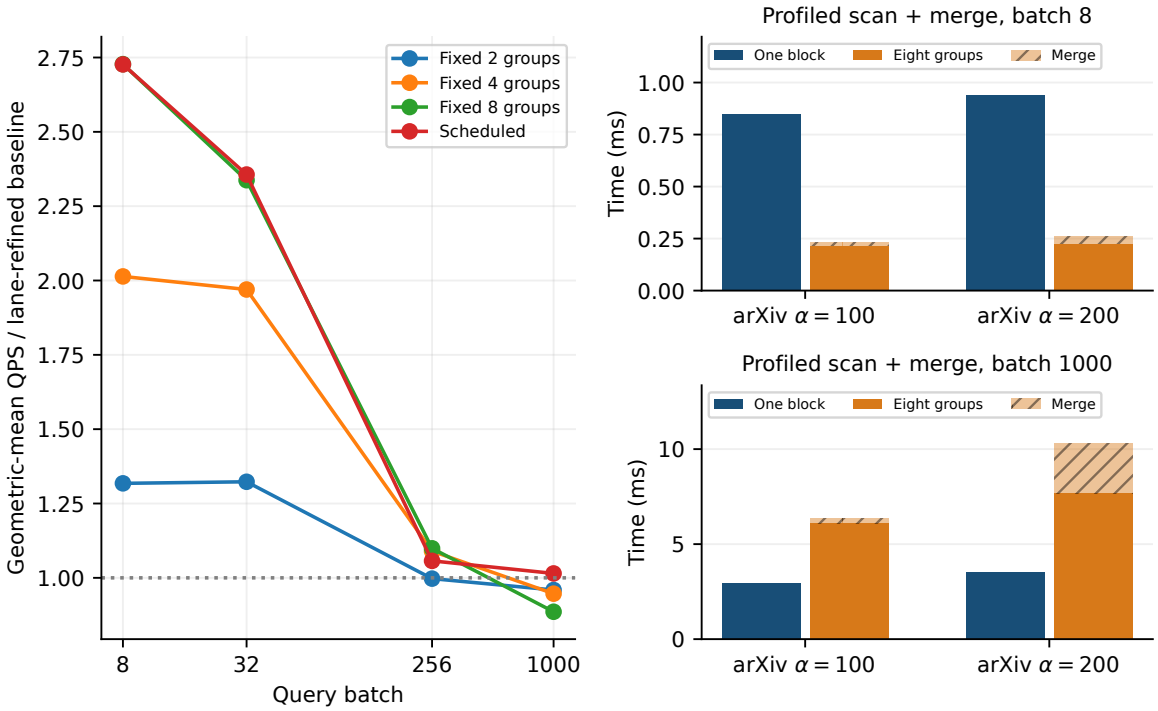


Figure 7.9. Execution regimes. Left: fixed shared-bound group counts and scheduled execution relative to the lane-refined one-block baseline (13 cells at small batches, 11 at batch 1,000). Right: paired scan and merge profiles at batches 8 and 1,000 for arXiv budgets 100 and 200. Hatching denotes merge time. The right panels use the independent-bound scan experiment; they explain a cost, not the scheduler's entire speedup.

7.6.2 A fixed rule evaluated on further configurations

The scheduler follows the fill/skew rule and pseudoquery calibration in section 5.5. Four configurations set the constants before the additional configurations were measured.

Table 7.12. Integrated execution gains over v61 L16. Entries are geometric means of per-cell median paired speedups from five rounds. "Held out" excludes the four configurations used to set the rule; it means held-out configurations, not the budget experiment's held-out query split.

Batch	Cells	$g_s = 2$	$g_s = 4$	$g_s = 8$	Scheduled	Held out
8	13	1.318	2.014	2.728	2.727	2.930
32	13	1.323	1.970	2.337	2.356	2.560
256	13	0.997	1.090	1.099	1.057	1.041
1000	11	0.959	0.947	0.886	1.015	1.012

The means use thirteen small-batch configurations and eleven full-batch ones (Stella has 999 queries); held-out subsets use nine/seven (table 7.12). At batch 1,000, eight groups give 0.886×; worst regret is 1.217 (BGE/256).

Vogue's CV 0.338–0.622 selects four groups at batch 1,000. Ratios span 0.895–1.175; $(n_{\text{probe}}, \alpha) = (16, 16), (16, 64), (32, 64)$ give 0.955, 0.895, 0.955. CV measures spread, not total work: it enables the 0.97 lead yet overgroups low-probe loads (section 7.2).

Across 250 variant/configuration pairs and 234,320 query comparisons, there are no non-tie primary differences or final-ID-set differences. Tied output order can differ. These are sequential synchronous throughput tests, not open-loop tail-latency tests.

7.7 Cold Construction, Resident Memory and Tested Scale

RQ1 is assessed through cold construction, resident memory and completed large-dataset searches; the E1 memory comparison also informs RQ4.

7.7.1 Cold builds and the lifecycle boundary

Three empty-cache repeats give Vogue 298–303 ms training and ≈ 0.56 s total, versus the slow archived 1.33 s. Stella takes 10.54–10.65 s versus archived 10.40 s (table 7.13).

Table 7.13. Cold JHQ construction and separately measured resident memory. Train/add are component medians of three supplementary normal builds; Total is the median paired sum. Memory comes from v71's two-group, 128-probe, $\alpha = 100$ runs and includes context and workspace. "Residual" is the modelled residual-code share of the measured used total, not the fraction read per query.

Dataset	Train s	Add s	Total s	Used GiB	Raw GiB	Residual
Vogue	0.30	0.26	0.56	1.47	2.67	45.4%
arXiv	0.48	0.51	0.99	2.58	6.45	62.5%
OpenAI-1536	0.61	0.38	1.00	2.39	5.72	59.7%
OpenAI-3072	1.16	0.67	1.82	4.14	11.43	69.0%
BGE	3.56	3.65	7.20	11.93	38.50	80.7%
Stella	3.55	6.99	10.55	20.44	67.81	82.9%

Sections 3.3 and 3.5.2 define startup/build boundaries; figure 7.10 compares repeated builds.

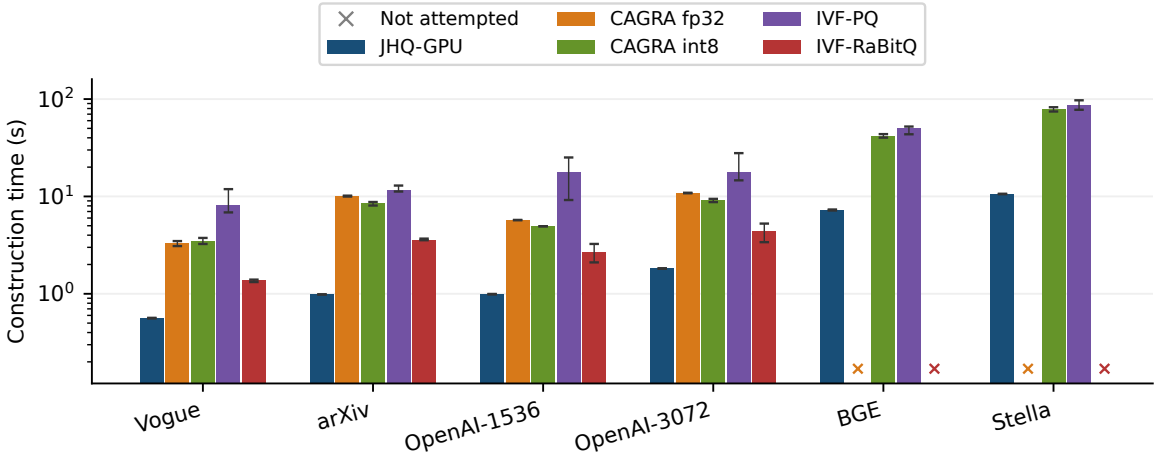


Figure 7.10. Recorded construction costs, with CAGRA int8 included. JHQ is the median and range of three supplementary normal cold builds with warm input pages; other bars are medians across successful measured index configurations, with minimum–maximum whiskers. Int8 includes dataset quantization; RaBitQ excludes the required serialization round trip. Missing large-dataset bars denote unattempted configurations, not measured build failures. The vertical axis is logarithmic.

Stella takes 74.55/82.45 s with isolated CAGRA int8 (quantisation included), and median 86.59 s across three IVF-PQ configurations, versus JHQ's 10.54–10.65 s. Quality/resident memory are separate comparisons.

Five arXiv builds span 0.0012 recall. Stella startup takes 18.2–18.6 s with warm pages, 129.9 s on first read. Sampled 20 ms peaks are BGE 26.08 GiB and Stella 22.58 GiB. BGE's 12 GiB assignment buffers precede its 23.05 GiB gather payload, so they must not be summed (equation (4.13)).

7.7.2 Resident allocation and where compression helps

Figure 7.11 combines the component model in section 4.4 with measured resident totals. Its unattributed remainder is retained, and the preceding construction measurements give the sampled peaks.

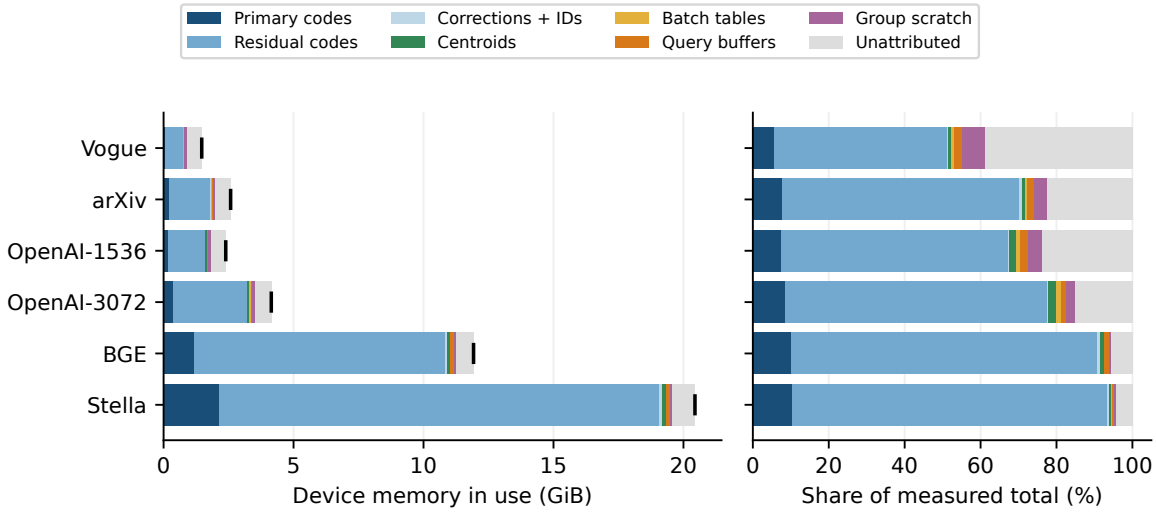


Figure 7.11. Resident-memory decomposition at two-group tables, 128 probes and $\alpha = 100$. Component sizes are modelled, markers are measured used totals, and hatching is the unattributed remainder. The totals include context and workspace. The preceding construction measurements give sampled peaks; these resident markers are not peaks.

Residuals occupy 80.7% of BGE's 11.93 GiB and 82.9% of Stella's 20.44 GiB totals, including context/workspace. Raw vectors need 38.50/67.81 GiB. Smaller α saves reads, not stored residual bytes (section 4.4).

The E1 measurements provide a separate cross-system view in table 7.14. Subtracting the baseline-measured 507.2 MiB context from JHQ's process total gives approximately 0.29–0.39 of CAGRA fp32's index allocation. JHQ's figure includes workspace, whereas CAGRA's is the build-time index delta. These are parameter-sweep ranges, not memory-matched recall points or a comparison of equal peak budgets. The CAGRA int8 allocations in the same table are of a similar order to JHQ's: compression relative to fp32 is shared by competing designs. The component model in table 4.1 explains this comparison.

Table 7.14. E1 allocation ranges. JHQ is process total minus a 507.2 MiB context estimate, including query workspace; both CAGRA columns are index allocations measured during build. Ranges cover the parameter sweep.

Dataset	JHQ net process GiB	CAGRA fp32 index GiB	CAGRA int8 index GiB
Vogue	0.90–1.01	2.89–2.91	0.89–0.91
arXiv	2.01–2.19	6.84–6.98	2.01–2.15
OpenAI-1536	1.79–2.33	5.95–5.97	1.67–1.68
OpenAI-3072	3.54–3.82	11.67–11.68	3.10–3.11

7.7.3 Large-dataset feasibility and competitive regions

BGE contains 10,091,524 vectors and Stella 17,776,615, both with 1,024 dimensions and 32,768 IVF lists. Stella's queries are sampled from the corpus; its result therefore charac-

terizes that benchmark rather than an independently collected external-query distribution. Figure 7.12 includes successful CAGRA int8 and IVF-PQ measurements. Full-precision CAGRA/RaBitQ/FP32 brute force were excluded where predicted storage exceeded capacity. These are planned exclusions; no out-of-memory or recall failure was measured.

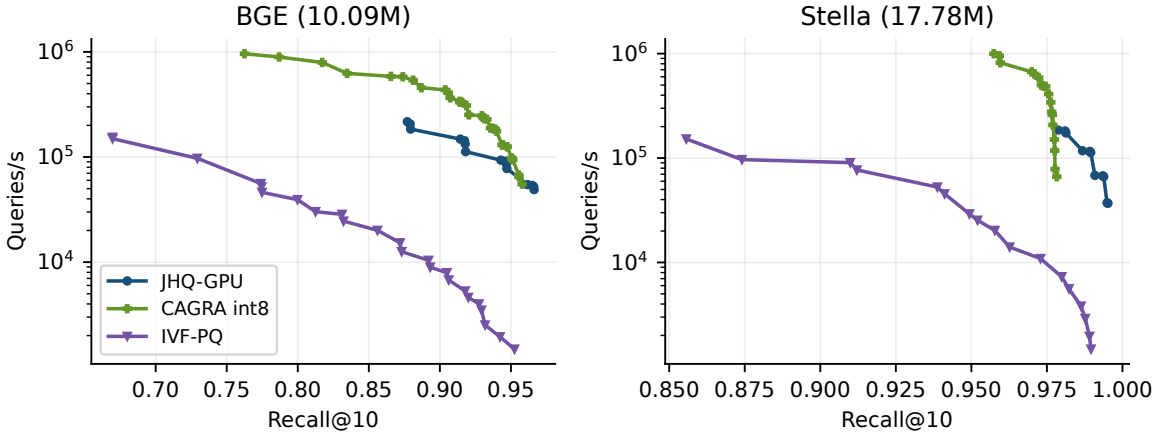


Figure 7.12. Large-dataset host-to-host frontiers, including the successful isolated CAGRA int8 reruns. BGE/Stella use 1,000/999 queries. Endpoints mark tested-grid coverage.

CAGRA int8 fits Stella. On BGE it leads 1.35–2.96× at recall 0.88–0.95; JHQ leads 1.086× at int8's maximum 0.9579. Maximum BGE/Stella recall: int8 0.9579/0.9783, JHQ 0.9663/0.9952, PQ 0.9525/0.9896.

Stella gives JHQ 185.4 kQPS at 0.978679 recall versus int8 66.3 at 0.978278. The 2.80× point ratio is not recall-matched; int8 lacks upper-region coverage (figure 7.12).

7.8 Numerical Checks and Recall Analysis

7.8.1 Quality resolution and repeated execution

A single GT hit changes Recall@10 by

$$\Delta r = \frac{1}{nk} = \begin{cases} 10^{-4}, & n = 1000, \\ 2 \times 10^{-4}, & n = 500. \end{cases}$$

Stella's $n = 499$ step is slightly larger. This step size is not an uncertainty interval.

All 369 E1 baseline repeat-ID flags stay, including tied reordering. The 401 convention adds cross-protocol differences (section 3.5.2); these are not independent builds.

Near 0.97, local slopes imply 1.01–3.17% QPS change/10^{-3} recall before snapping. Five-build recall ranges are Vogue/arXiv/OpenAI-1536/3072: 0.0008/0.0012/0.0004/0.0002. This suggests local sensitivity, not a retrained-frontier bound. Cached-model FP32 checks retain OpenAI's advantage (section 7.2.3); arXiv/LUT near ties remain inconclusive.

RaBitQ's median five-round ranges are 3.79%, 3.34%, 2.87%, 5.25% of median QPS on Vogue/arXiv/OpenAI-1536/3072. These spreads are not confidence intervals. To reduce occasional 30–60 ms calls' influence, we recompute the 1,000-query rate as 10^6/ median(call ms). At recall 0.97, the resulting JHQ/RaBitQ ratios are 1.063, 0.995, 1.118 and 1.100. The three favourable directions and arXiv near parity remain, so the result is not solely a consequence of long calls.

7.8.2 Reconstructed scores and recall loss

The stage contracts in table 5.1 distinguish computed-score selection from raw-space kNN; lane rounding, cutoff ties and process variation qualify different stages.

The archived reconstruction pilot measures the remaining gap to full-probe reconstructed scoring. Across seven workload–budget arms of 128 queries each, scoring all probed candidates

in FP64 improves recall over GPU outputs at selected budgets by 0–0.002344. For arXiv at 128 probes, $\alpha = 200$ matches the full-IVF reconstructed answer set on all 128 queries. At 100, the recall is 0.9734375 versus 0.9750 for full-IVF scoring: an additional 0.0015625 budget deficit beside the joint routing/reconstruction deficit of 0.0250 (section 6.2). These pilot aggregates include two budgets on one dataset–probe workload. FP64/GPU arithmetic differences prevent pure budget attribution in every arm. Separating routing/representation needs stage-wise GT coverage.

8 Discussion and Future Work

8.1 Design Implications

8.1.1 Codebook structure and lookup cost

Mailman, a method that reuses sums for finite-alphabet codes, gains no speed here; two-group LUTs usually win (figure 7.4). Codebook quality must therefore be evaluated with achievable table placement/access cost (sections 5.2 and 5.3).

8.1.2 Parallelism and selection overhead

Once queries supply enough parallelism, staging/selection/merge overhead can dominate (figure 7.9). Vogue/BGE expose missing absolute-work/decomposition features (sections 7.2 and 7.6). Richer scheduling must repay calibration; bounds alone are not isolated.

8.1.3 Refinement speed and budget savings

Assume fixed $T_0 > 0$, refinement times $F_h \geq F_\ell > 0$ and common multiplier $\eta \in (0, 1]$. The low-budget speedup and its derivatives are

$$S(\eta, T_0) = \frac{T_0 + \eta F_h}{T_0 + \eta F_\ell}, \tag{8.1}$$
$$\frac{\partial S}{\partial \eta} = \frac{T_0(F_h - F_\ell)}{(T_0 + \eta F_\ell)^2} \geq 0, \qquad \frac{\partial S}{\partial T_0} = \frac{\eta(F_\ell - F_h)}{(T_0 + \eta F_\ell)^2} \leq 0.$$

For $F_h > F_\ell$, faster refinement reduces the value of cutting its budget; lower fixed cost increases it. The derivatives hold with non-refinement work fixed. Budget-dependent scan maintenance and resource transitions require the controlled comparisons of sections 7.5 and 7.5.6. Quality and reuse remain constraints (table 7.8 and figure 7.6).

8.2 Alternative Scoring and Coding Experiments

8.2.1 Mailman-style shared-sum scoring

Mailman reuses finite-alphabet sums [20]. Frozen Vogue/arXiv/OpenAI-3072 streams compare atomic buckets/private histograms at widths 4/6/8 with packed two-table scoring: 128 probes, batches 8/1,000, fixed codes/mathematical scores.

The best alternatives reach only 0.187–0.581 of baseline QPS (figure 8.1). Biased bucket destinations dominate profiles; private histograms add state/shared updates. Narrower groups sometimes help, but none wins. Other finite-alphabet designs remain open.

Only scores were tested, so cutoff checks do not establish neighbour-pipeline equivalence. The replacement model gives

$$\frac{T'}{T} = 1 + f\left(\frac{1}{s} - 1\right) > 1 \quad (0 < f \leq 1,\ 0 < s < 1).$$

This model is not measured end-to-end performance; integration stopped. A dictionary study also stopped at structural statistics. Neither contributes a system-frontier point.

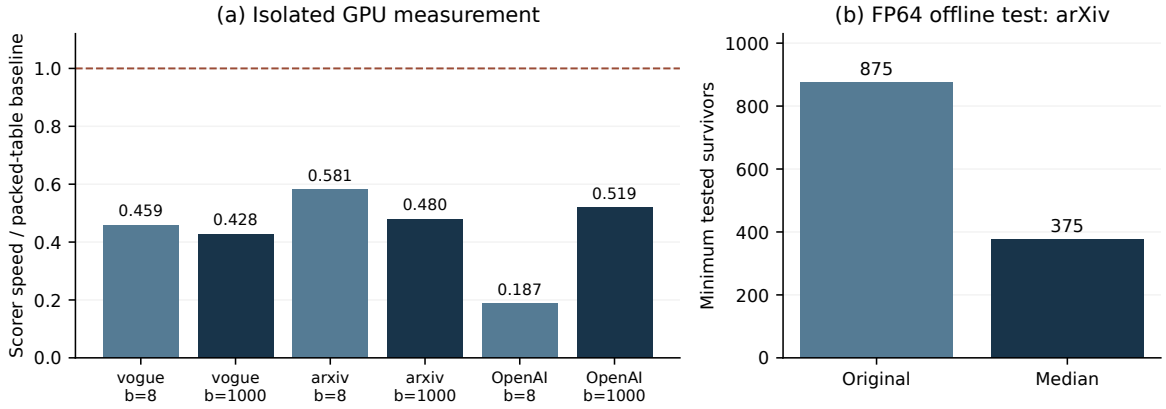


Figure 8.1. Two distinct evidence boundaries. Left: best measured Mailman scorer / packed-table scorer speed on an RTX 5090, with Vogue, arXiv and OpenAI-3072 at two batches and 128 probes; all ratios are below one. Right: minimum survivor counts meeting the original JHQ recall target on arXiv's 400-query FP64 offline test split. The candidate reduction is not a GPU speedup.

8.2.2 Rank-oriented coding experiments

FP64 candidate replay uses 400/200/400 calibration/validation/test queries. It tests weighted scores, median/list thresholds, learned losses and linear minimum mean-square-error (MMSE) decoding. Lower MSE can worsen neighbour order; primary-level changes also move residual reference points.

Median thresholds reduce arXiv's minimum tested qualifying count 875→375; two other workloads fail. For orthogonal Π,

$$\bar{y} = \frac{1}{N}\sum_i \Pi x_i = \Pi \bar{x}, \qquad \|\bar{y}\|^2 = \bar{x}^\top \Pi^\top \Pi \bar{x} = \|\bar{x}\|^2.$$

Rotation therefore preserves mean-vector norm rather than removing bias. Centring must also preserve the refined-score ceiling.

Median-level GPU follow-up retrains residuals. ArXiv's original $\alpha = 100$ target gains 1.40/1.21× at batch 8/1,000; Vogue reaches 0.9630 below 0.9644, OpenAI-3072 ties. Three rounds use separate states/queries, outside E1.

8.3 Threats to Internal and External Validity

Table 8.1 consolidates threats to the main interpretations. Controls are in Chapter 3; repeat/build/precision checks are in section 7.8.

Table 8.1. Threats that can reverse or narrow a conclusion.

Claim at risk	Alternative or reversal	Control and remaining test
Execution gain	Changed scores, ties or index state explain an apparent speedup	Strict T4 CPU sorting and table-form diagnostics; finite coverage, with candidate dumps no longer retained
Budget quality	Finite-pool sampling or drift hides damaging queries	Disjoint GT evaluation, full traces, subset-sum checks and FULL enumeration; future-query and arbitrary-subset invariance remain untested
Competitive margin	Grid density, call tails or API overhead changes the ordering	Both recall estimators, retained slow calls and int8 baseline; TF32/FP32 sensitivity; T5 exposes small-batch reversals; cross-device comparisons remain unmeasured
Capacity/build claim	Transient allocations exceed a resident sample	Phase bytes, repeated empty-cache builds and 20 ms device peaks; sub-interval allocations and full retrained frontiers remain unmeasured

Historical CPU JHQ runs use different index parameters and recall/timing protocols from E1 (section 7.1). Their throughput cannot establish a matched-quality CPU/GPU speedup; that comparison remains unmeasured.

Evidence covers one GPU, six prepared datasets and squared-L2 top-10 (section 7.2.6). Near duplicates may remain; relevance labels are absent. Updates, concurrency, drift, service latency, billion-scale and multi-device use are untested.

8.4 Future Research Directions

8.4.1 Calibration risk and reuse cost

Future cost-aware calibration needs valid query-level bounds, matched-regime counts, fixed reference, sampling/order assumptions and drift renewal. Absolute GT risk additionally needs labels or a justified reference-to-GT link.

Compare sampling/rank reuse/cost-aware choice by held-out total time and tail agreement. A four-cell sequential controller lost to extra launches. A replacement must beat a quality-matched control while meeting its risk bound.

8.4.2 Residual storage across GPU and host memory

A future residual cache could fetch host blocks under a device cap. Fix representation, routing, survivors and arithmetic; vary cap, batch and locality; measure bytes, hits, overlap, tails and QPS. This capacity study is unimplemented; lowering α does not relocate stored residuals.

8.5 Deployment Considerations

For recall 0.97, E1 supports batch-1,000 throughput; at batch 8, T5 favours CAGRA fp32. Batch-32 OpenAI-3072 depends on the estimator (section 7.2.6). Begin with FULL, then validate quality/reuse cost/memory (section 8.3).

9 Conclusion

9.1 Answers to the Four Research Questions

RQ1: GPU index construction and capacity. GPU construction produces a searchable resident hierarchy. The largest test holds 17.78 million vectors at 20.44 GiB, built in 10.54–10.65 s; section 7.7 reports construction peaks.

RQ2: Representation-aware execution. Cartesian splitting preserves real-arithmetic scores; complete-key retention/merge and witnessed bounds preserve grouped selection under their premises. Two-group LUTs win 22/24 cells. Scheduling gains 2.727/1.015× at batch 8/1,000 against lane refinement (sections 7.3 and 7.6). Strict CPU checks confirm grouped order; one-block differences are cutoff ties (section 7.4.1).

RQ3: Workload-level refinement calibration. FULL calibration meets the tested relative recall-loss target in 12/12 cells, with maximum observed loss 0.0006. $S = 128$ reaches 40/64 crossings in one cell (section 7.5). FULL matches enumeration; ten distance-only cells lack general nesting. Reference agreement is not population GT recall. Four direct reuse tests show workload-dependent quality/time gains.

RQ4: Cross-system search performance. At batch 1,000/recall 0.97, JHQ leads on three datasets and nearly ties RaBitQ on arXiv; Vogue's lead needs grouping (section 7.2). OpenAI E1 uses TF32. FP32 checks retain gains on the TF32-trained cache (section 7.2.4). Int8/RaBitQ win other recall regions; CAGRA fp32 wins all batch-8 datasets. Batch-32 near ties depend on the estimator (section 7.2.6).

9.2 Summary and Outlook

JHQ's Cartesian structure is useful when the execution path makes its small tables and packed codes cheap to access. Parallel decomposition then depends on workload: grouping exposes work for small batches, while large batches can already provide enough independent queries. Refinement calibration adds a separate quality decision, since agreement on a finite query pool does not ensure future recall. The next step is to join risk-controlled calibration with measured reuse cost, then study residual placement when device memory becomes the limiting resource (section 8.4).

Bibliography

[1] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. Cache locality is not enough: High-performance nearest neighbor search with product quantization fast scan. *Proc. VLDB Endow.*, 9(4):288–299, 2015. doi: 10.14778/2856318.2856324.

[2] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. Quicker ADC: Unlocking the hidden potential of product quantization with SIMD. *IEEE Trans. Pattern Anal. Mach. Intell.*, 43 (5):1666–1677, 2021. doi: 10.1109/TPAMI.2019.2952606.

[3] Anastasios N. Angelopoulos, Stephen Bates, Emmanuel J. Candès, Michael I. Jordan, and Lihua Lei. Learn then test: Calibrating predictive algorithms to achieve risk control, 2022. URL `https://arxiv.org/abs/2110.01052v5`. Version 5; Section 2.3.1.

[4] Anthropic. Claude: Implementation, experiment and review assistance record. Project AI-use record, 2026. Implementation and experiment records; original prompts and interaction metadata unavailable.

[5] Tony Assi. Vogue933k embeddings. Hugging Face Datasets, n.d. URL `https://huggingface.co/datasets/tonyassi/vogue933k-embeddings`. Upstream attribution recorded in the project data preparation source.

[6] Manos Chatzakis, Yannis Papakonstantinou, and Themis Palpanas. DARTH: Declarative recall through early termination for approximate nearest neighbor search. *Proc. ACM Manag. Data*, 3 (4):242:1–242:26, 2025. doi: 10.1145/3749160. URL `https://arxiv.org/abs/2505.19001`.

[7] Artem Chirkin, Akira Naruse, Tamás Fehér, Yong Wang, and Corey Nolet. Accelerating vector search: NVIDIA cuVS IVF-PQ part 2, performance tuning, 2024. URL `https://developer.nvidia.com/blog/accelerating-vector-search-nvidia-cuvs-ivf-pq-performance-tuning-part-2/`. 18 July; accessed 28 October 2026.

[8] FAISS contributors. FAISS clustering parameters, version 1.9.0. `https://github.com/facebookresearch/faiss/blob/v1.9.0/faiss/Clustering.h`, 2024. Default minimum-points warning threshold; accessed 28 October 2026.

[9] FAISS contributors. *Index I/O, Cloning and Hyper-parameter Tuning*, n.d.. URL `https://github.com/facebookresearch/faiss/wiki/Index-IO,-cloning-and-hyper-parameter-tuning`. Auto-tuning the runtime parameters; accessed 28 October 2026.

[10] FAISS contributors. *IndexRefine: Candidate Expansion and Refinement*, n.d.. URL `https://faiss.ai/cpp_api/file/IndexRefine_8h.html`. C++ API documentation; accessed 28 October 2026.

[11] Jianyang Gao and Cheng Long. RaBitQ: Quantizing high-dimensional vectors with a theoretical error bound for approximate nearest neighbor search, 2024. URL `https://arxiv.org/abs/2405.12497`. SIGMOD 2024 paper.

[12] Jianyang Gao, Yutong Gou, Yuexuan Xu, Yongyi Yang, Cheng Long, and Raymond Chi-Wing Wong. Practical and asymptotically optimal quantization of high-dimensional vectors in euclidean space for approximate nearest neighbor search, 2024. URL `https://arxiv.org/abs/2409.09913`.

[13] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. Optimized product quantization for approximate nearest neighbor search. In *IEEE CVPR*, pages 2946–2953, 2013. doi: 10.1109/CVPR.2013.379. URL `https://openaccess.thecvf.com/content_cvpr_2013/html/Ge_Optimized_Product_Quantization_2013_CVPR_paper.html`.

[14] Oded Green, Robert McColl, and David A. Bader. GPU Merge Path: A GPU merging algorithm. In *Proceedings of the 26th ACM International Conference on Supercomputing*, pages 331–340, 2012. doi: 10.1145/2304576.2304621. URL `https://davidbader.net/publication/2012-gm-ba/2012-gm-ba.pdf`.

[15] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. Accelerating large-scale inference with anisotropic vector quantization. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 3887–3896, 2020. URL `https://proceedings.mlr.press/v119/guo20h.html`.

[16] Jiabao Han, Mengxuan Zhang, and Goce Trajcevski. JHQ: Johnson-Lindenstrauss enhanced hierarchical quantization for high-dimensional approximate nearest neighbor search. *Proc. VLDB Endow.*, 19(7):1530–1543, 2026. doi: 10.14778/3801059.3801067.

[17] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Trans. Pattern Anal. Mach. Intell.*, 33(1):117–128, 2011. doi: 10.1109/TPAMI.2010.57.

[18] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs, 2017. URL `https://arxiv.org/abs/1702.08734`.

[19] Conglong Li, Minjia Zhang, David G. Andersen, and Yuxiong He. Improving approximate nearest neighbor search through learned adaptive early termination. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, 2020. doi: 10.1145/3318464.3380600. URL `https://www.pdl.cmu.edu/PDL-FTP/BigLearning/mod0246-liA.pdf`.

[20] Edo Liberty and Steven W. Zucker. The Mailman algorithm: A note on matrix–vector multiplication. *Information Processing Letters*, 109(3):179–182, 2009. doi: 10.1016/j.ipl.2008.09.028. URL `https://www.cs.yale.edu/publications/techreports/tr1402.pdf`.

[21] Yu. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. arXiv:1603.09320, 2016. URL `https://arxiv.org/abs/1603.09320`. Version 4, revised August 2018.

[22] National Institute of Standards and Technology. *Dataplot: Exact Binomial Confidence Limits*, n.d.. URL `https://itl.nist.gov/div898/software/dataplot/refman2/auxillar/exacbino.htm`. One-sided exact binomial limits; accessed 28 October 2026.

[23] National Institute of Standards and Technology. *Dataplot: Hypergeometric Probability Density Function*, n.d.. URL `https://www.itl.nist.gov/div898/software/dataplot/refman2/ch8/hyppdf.pdf`. Sampling without replacement; accessed 28 October 2026.

[24] NVIDIA. *NVIDIA Blackwell Tuning Guide*, 2025. URL `https://docs.nvidia.com/cuda/archive/13.0.1/blackwell-tuning-guide/index.html`. Section 1.4.1.1; accessed 28 October 2026.

[25] NVIDIA. *CUDA C++ Programming Guide, Release 13.0.1*. NVIDIA, 2025. URL `https://docs.nvidia.com/cuda/archive/13.0.1/cuda-c-programming-guide/index.html`. Sections 10.14.1.4 and 18.5.3.3; accessed 27 October 2026.

[26] NVIDIA. *PTX ISA, Version 9.0*, 2025. URL `https://docs.nvidia.com/cuda/archive/13.0.1/parallel-thread-execution/index.html`. Sections 8.1, 8.4.2, 8.7 and 8.10.3; accessed 28 October 2026.

[27] NVIDIA. *CUDA Programming Guide: The CUDA Platform*, 2026. URL `https://docs.nvidia.com/cuda/cuda-programming-guide/01-introduction/cuda-platform.html`. PTX compatibility and just-in-time compilation; accessed 28 October 2026.

[28] Hiroyuki Ootomo, Akira Naruse, Corey Nolet, Ray Wang, Tamas Feher, and Yong Wang. CAGRA: Highly parallel graph construction and approximate nearest neighbor search for GPUs. In *IEEE ICDE*, 2024. doi: 10.1109/ICDE60146.2024.00323.

[29] OpenAI. ChatGPT: Project review assistance record. Project AI-use record, 2026. Retained review excerpts; original prompts and interaction metadata unavailable.

[30] OpenAI. Codex: Selected thesis-preparation interactions. Project AI-use record, 2026. Selected prompts and uses; compiled 29 October 2026. Partial record; historical interaction metadata unavailable.

[31] Qdrant. DBpedia entities: OpenAI text-embedding-3-large, 1536 dimensions, 1m. Hugging Face Datasets, n.d.. URL `https://huggingface.co/datasets/Qdrant/dbpedia-entities-openai3-text-embedding-3-large-1536-1M`. Accessed 28 October 2026.

[32] Qdrant. DBpedia entities: OpenAI text-embedding-3-large, 3072 dimensions, 1m. Hugging Face Datasets, n.d.. URL `https://huggingface.co/datasets/Qdrant/dbpedia-entities-`

`openai3-text-embedding-3-large-3072-1M`. Accessed 28 October 2026.

[33] Jifan Shi, Jianyang Gao, James Xia, Tamás Béla Fehér, and Cheng Long. GPU-native approximate nearest neighbor search with IVF-RaBitQ: Fast index build and search. *Proc. VLDB Endow.*, 19(11):3454–3467, 2026. doi: 10.14778/3836663.3836701.

[34] Suhas Jayaram Subramanya, Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnaswamy, and Rohan Kadekodi. DiskANN: Fast accurate billion-point nearest neighbor search on a single node. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL `https://proceedings.neurips.cc/paper_files/paper/2019/hash/09853c7fb1d3f8ee67a61b6bf4a7f8e6-Abstract.html`.

[35] The Alexandria Index. arXiv abstracts dataset. Hugging Face Datasets, n.d. URL `https://huggingface.co/datasets/macrocosm/arxiv_abstracts`. Accessed 28 October 2026; exported embedding preparation is project-specific.

[36] The Information Engineering Lab. Stella TREC24 BioGen embedding. Hugging Face Datasets, n.d. URL `https://huggingface.co/datasets/ielabgroup/stella_trec24_biogen_embedding`. Corpus split; accessed 28 October 2026.

[37] Upstash. Wikipedia 2024-06 BGE-M3. Hugging Face Datasets, n.d. URL `https://huggingface.co/datasets/Upstash/wikipedia-2024-06-bge-m3`. Italian configuration; accessed 28 October 2026.

[38] Karthik Venkatasubba, Saim Khan, Somesh Singh, Harsha Vardhan Simhadri, and Jyothi Vedurada. BANG: Billion-scale approximate nearest neighbour search using a single GPU. *IEEE Transactions on Big Data*, 11(6):3142–3157, 2025. doi: 10.1109/TBDATA.2025.3581085. URL `https://ssomesh.github.io/docs/papers/tbd25.pdf`.

[39] Jingrong Zhang, Akira Naruse, Xipeng Li, and Yong Wang. Parallel top-k algorithms on GPU: A comprehensive study and new methods. In *SC '23*, 2023. doi: 10.1145/3581784.3607062. URL `https://sc23.supercomputing.org/proceedings/tech_paper/tech_paper_pages/pap294.html`.

[40] Weijie Zhao, Shulong Tan, and Ping Li. SONG: Approximate nearest neighbor search on GPU. In *IEEE ICDE*, pages 1033–1044, 2020. doi: 10.1109/ICDE48307.2020.00094. URL `https://www.cs.rit.edu/~wjz/papers/conference/2020-icde-song.pdf`.